

Relational Algebra - 1

DBMS Lecture Series

Unit-3

Pavinder Yadav

Assistant Professor
School of Computer Science
University of Petroleum and Energy Studies
Dehradun



- 1 Introduction to Relational Algebra
- 2 Selection (σ) Operator
- 3 Projection (π) Operator
- 4 Set operators: Union ($\cup$), Intersection ($\cap$), Difference ($-$)

- 1 Introduction to Relational Algebra
- 2 Selection (σ) Operator
- 3 Projection (π) Operator
- 4 Set operators: Union ($\cup$), Intersection ($\cap$), Difference ($-$)

- **Relational Algebra** is a **procedural query language** which:
 - Takes a **relation** as input
 - Produces another **relation** as output
- It is a language for expressing **relational database queries**.
- Queries are expressed using **operators** (either **unary** or **binary**).
- **Types of operators in Relational Algebra:**
 - 1 **Basic / Fundamental Operators**
 - 2 **Additional / Derived Operators**
- Relational algebra operations:
 - Work on one or more relations to define new relations
 - Do not modify the original relations
 - Are performed **recursively** on relations

Key Idea

Relational Algebra = **Procedural way** of defining queries using operators.

- In Relational Algebra:
 - **Input** is a **relation** (table).
 - **Output** is also a **relation**.
- Operations are recursive: intermediate results are relations too.
- Works on the whole relation at once (no loops required).
- We only need to specify the **table name** and condition.

Student Relation

ID	Name	Age
1	Akon	17
2	Bkon	19
3	Ckon	15
4	Dkon	13

Query:

Select name of the students with age less than 17.

$$\pi_{\text{Name}}(\sigma_{\text{Age} < 17}(\text{Student}))$$

Output:

Name
Ckon
Dkon

1. Basic / Fundamental Operations:

- ① Selection (σ)
- ② Projection (π)
- ③ Union ($\cup$)
- ④ Set Difference ($-$)
- ⑤ Cartesian Product ($\times$)
- ⑥ Rename (ρ)

Note

- σ, π, ρ are **Unary Operators** (work on one relation).
- $\cup, -, \times$ are **Binary Operators** (work on two relations).

2. Additional / Derived Operations:

- ① Natural Join ($\bowtie$)
- ② Outer Joins: Left, Right, Full ($\ltimes, \rtimes, \bowtie$)
- ③ Set Intersection ($\cap$)
- ④ Division ($\div$)
- ⑤ Assignment ($\leftarrow$)

Note

All derived operations are **Binary Operators**.

- 1 Introduction to Relational Algebra
- 2 Selection (σ) Operator**
- 3 Projection (π) Operator
- 4 Set operators: Union ($\cup$), Intersection ($\cap$), Difference ($-$)

- The **Selection Operator** (σ) is a **unary operator** in Relational Algebra.
- It selects **tuples (rows)** that satisfy a given **condition (predicate)** from a relation.
- Denoted by: σ

Notation

$$\sigma_p(R) \quad \text{or} \quad \sigma_{\text{Condition}}(\text{Relation})$$

where p is a propositional formula using:

- Logical connectives: $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT)
- Relational operators: $=, \neq, <, >, \leq, \geq$

SQL Correspondence

Relational Algebra selection corresponds to the **WHERE clause** in SQL.

```
SELECT * FROM R WHERE Condition;
```

Examples

Example: Select students with age > 17:

$$\sigma_{\text{Age} > 17}(\text{Student})$$

Query: Select student whose roll number = 2

$$\sigma_{\text{roll_no}=2}(\text{Student})$$

Student Relation:

roll_no	name	age	address
1	A	20	Bhopal
2	B	17	Mumbai
3	C	16	Mumbai
4	D	19	Delhi
5	E	18	Delhi

Result:

roll_no	name	age	address
2	B	17	Mumbai

Note

In Selection, the schema of result is identical to the input relation.

Query: Select students whose age > 17 AND who live in Delhi

$$\sigma_{\text{age} > 17 \wedge \text{address} = \text{"Delhi"}} (\text{Student})$$

Student Relation:

roll_no	name	age	address
1	A	20	Bhopal
2	B	17	Mumbai
3	C	16	Mumbai
4	D	19	Delhi
5	E	18	Delhi

Result:

roll_no	name	age	address
4	D	19	Delhi
5	E	18	Delhi

- **Case 1:** Subject = "database"

$$\sigma_{\text{subject}=\text{"database"}}(\text{Books})$$

- **Case 2:** Subject = "database" AND Price = 450

$$\sigma_{\text{subject}=\text{"database"} \wedge \text{price}=450}(\text{Books})$$

- **Case 3:** Subject = "database" AND Price = 450 OR Year > 2010

$$\sigma_{\text{subject}=\text{"database"} \wedge \text{price}=450 \vee \text{year}>2010}(\text{Books})$$

Observation

Selection conditions can be combined with **logical connectives** $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT).

- 1 Introduction to Relational Algebra
- 2 Selection (σ) Operator
- 3 Projection (π) Operator**
- 4 Set operators: Union ($\cup$), Intersection ($\cap$), Difference ($-$)

- The **Projection Operator** (π) is a **unary operator**.
- It **projects (displays) particular columns (attributes)** from a relation.
- Removes the columns not in the projection list.
- Duplicate rows are automatically eliminated.

Notation

$$\pi_{A_1, A_2, \dots, A_n}(R) \quad \text{or} \quad \pi_{\text{Attribute_list}}(\text{Relation})$$

where $A_1, A_2, \dots, A_n$ are attribute names of relation R .

SQL Correspondence

Projection in Relational Algebra $\equiv$ `SELECT column_list FROM Table;`

The **SQL SELECT command** corresponds to relational projection π . General SQL form:

SELECT A1, A2, ..., An FROM R;

Example

Display the columns roll_no and name from relation **Student**.

$\pi_{\text{roll_no, name}}(\text{Student})$

Student Relation:

roll_no	name	age	address
1	A	20	Bhopal
2	B	17	Mumbai
3	C	16	Mumbai
4	D	19	Delhi
5	E	18	Delhi

Result:

roll_no	name
1	A
2	B
3	C
4	D
5	E

Query 1: Display (or project) the name of students in Student table.

$\pi_{\text{name}}(\text{Student})$

Student Relation:

roll_no	name	age
1	A	20
2	B	17
3	C	16
4	D	19
5	E	18
6	F	18

Result:

name
A
B
C
D
E
F

Query 2: Display the roll_no and name of students in Student table.

$$\pi_{\text{roll_no, name}}(\text{Student})$$

Student Relation:

roll_no	name	age
1	A	20
2	B	17
3	C	16
4	D	19
5	E	18
6	F	18

Result:

roll_no	name
1	A
2	B
3	C
4	D
5	E
6	F

Query 3: Display the roll_no and name of students whose age > 17.

$$\pi_{\text{roll_no, name}}(\sigma_{\text{age} > 17}(\text{Student}))$$

Student Relation:

roll_no	name	age
1	A	20
2	B	17
3	C	16
4	D	19
5	E	18
6	F	18

Result:

roll_no	name
1	A
4	D
5	E
6	F

Note

Applying **Projection after Selection** is correct. Doing it the other way round (Selection on projected attributes not present) is wrong.

- 1 Introduction to Relational Algebra
- 2 Selection (σ) Operator
- 3 Projection (π) Operator
- 4 Set operators: Union ($\cup$), Intersection ($\cap$), Difference ($-$)**

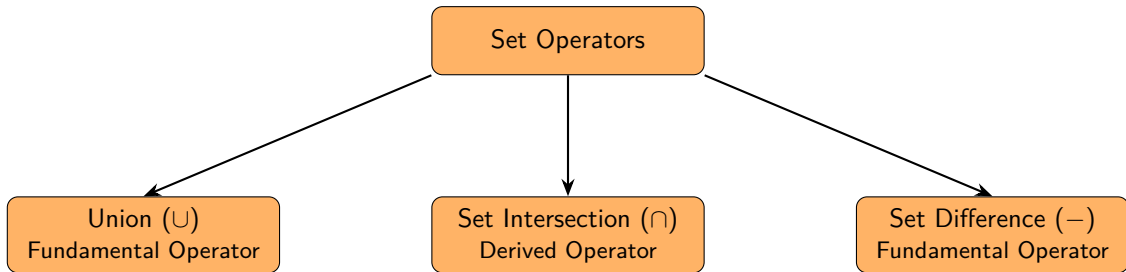
- **Set operators:** Union ($\cup$), Intersection ($\cap$), Difference ($-$).
- These are **binary** operators (take two input relations).
- To use set operators on two relations, the relations must be **compatible**.

Compatibility (a.k.a. Union-Compatibility)

Two relations R and S are compatible if:

- 1 R and S have the **same number of attributes**.
- 2 Corresponding attributes have the **same domains (types)**.

Duplicate tuples in results are **eliminated automatically**.



- For relations R and S , **Union** returns all tuples that are in R or S or both.
- Removes duplicates.

Validity Conditions (Compatibility)

- 1 R and S have the **same number** of attributes.
- 2 Corresponding attributes have the **same domains**.

Notation and SQL

Relational Algebra: $R \cup S$

SQL (set UNION eliminates duplicates by default):

```
(SELECT * FROM R) UNION (SELECT * FROM S);
```

Union Example 1 (Two-Column Relations)

Student

Roll_no	Name
1	A
2	B
3	C
4	D

Employee

Emp_no	Name
2	B
8	G
9	H

Result: (Student) $\cup$ (Employee)

Roll_no	Name
1	A
2	B
3	C
4	D
8	G
9	H

Examples

Assume Roll_no and Emp_no share the same domain (e.g., integer IDs). Union works by *position* when arity and domains match.

Goal: List all distinct names appearing in either Student or Employee.

$$\pi_{\text{Name}}(\text{Student}) \cup \pi_{\text{Name}}(\text{Employee})$$

Student

Roll_no	Name
1	A
2	B
3	C
4	D

Employee

Emp_no	Name
2	B
8	G
9	H

Result:

Name
A
B
C
D
G
H

- Suppose R and S are two relations.
- The **Set Intersection operation** selects all the tuples that are in **both** relations R and S .
- **Conditions for validity:**
 - 1 R and S must have the same number of attributes.
 - 2 Corresponding attributes must have the same domain (or type).

Notation and SQL

Relational Algebra: $R \cap S$

SQL (set Intersection eliminates duplicates by default):

```
(SELECT * FROM R) INTERSECTION (SELECT * FROM S);
```

Example: Intersection of Student and Employee

Student Table:

Roll_no	Name
1	A
2	B
3	C
4	D

Employee Table:

Emp_no	Name
2	B
8	G
9	H

- Result of $(Student) \cap (Employee)$:

Roll_no	Name
2	B

Note: Intersection is commutative, i.e., $A \cap B = B \cap A$

- Selects tuples that are present in relation R but not in relation S .
- Valid only if R and S have:
 - 1 Same number of attributes.
 - 2 Same domain for corresponding attributes.
- **Symbol:** — **Notation in Relational Algebra:** $R - S$
- **SQL Equivalent:** $(SELECT * FROM R) EXCEPT (SELECT * FROM S);$



Example: Consider two relations Student and Employee. We want to find tuples present in Student but not in Employee.

Student

Roll_no	Name
1	A
2	B
3	C
4	D

Employee

Emp_no	Name
2	B
8	G
9	H

Result: (Student – Employee)

Roll_no	Name
1	A
3	C
4	D

Note:

- Set Difference is **non-commutative**: $A - B \neq B - A$
- Intersection can be derived: $R - (R - S) = R \cap S$

Example: Display the names of students that are not present in Employee.

$$\Pi_{\text{Name}}(\text{Student}) - \Pi_{\text{Name}}(\text{Employee})$$

Student

Roll_no	Name
1	A
2	B
3	C
4	D

Employee

Emp_no	Name
2	B
8	G
9	H

Result

Name
A
C
D

- Find the names of the authors who have written books but not articles.

$$\Pi_{\text{author}}(\text{Books}) - \Pi_{\text{author}}(\text{Articles})$$

- Find the names of the authors who have written both a book and an article.

$$\Pi_{\text{author}}(\text{Books}) \cap \Pi_{\text{author}}(\text{Articles})$$

- Find the names of the authors who have either written a book or an article or both.

$$\Pi_{\text{author}}(\text{Books}) \cup \Pi_{\text{author}}(\text{Articles})$$

- Combines tuples from two relations using a condition.
- Types: Theta Join, Equi-Join, Natural Join.
- Notation: $R \bowtie_{\text{condition}} S$

Example

$\text{Employee} \bowtie_{\text{Employee.DeptID} = \text{Department.DeptID}} \text{Department}$

Output: Employees with department names.

- Finds tuples in R that are related to **all** tuples in S .
- Notation: $R \div S$

Example

Relation: **Course_Enroll(Student, Course)** Relation: **All_Courses(Course)**

$\text{Course_Enroll} \div \text{All_Courses}$

Output: Students enrolled in **all courses**.

Selection (σ)

- Unary operator $\rightarrow$ works on **rows (tuples)**.
- Selects tuples satisfying a given condition (predicate).
- Notation: $\sigma_{\text{condition}}(R)$ **Example:** $\sigma_{\text{age} > 17}(\text{Student})$

Projection (Π)

- Unary operator $\rightarrow$ works on **columns (attributes)**.
- Displays selected attributes, removes others.
- Duplicate rows are automatically eliminated.
- Notation: $\Pi_{\text{attributes}}(R)$ **Example:** $\Pi_{\text{name, roll_no}}(\text{Student})$

Set Operators

- **Union ($\cup$):** Returns tuples in R or S or both. Notation: $R \cup S$
- **Intersection ($\cap$):** Returns tuples common to both R and S . Notation: $R \cap S$
- **Difference ($-$):** Returns tuples in R but not in S . Notation: $R - S$
- All require relations to be **union-compatible**.

Thank You!



Relational Algebra - 2

DBMS Lecture Series

Unit-3

Pavinder Yadav

Assistant Professor
School of Computer Science
University of Petroleum and Energy Studies
Dehradun



- 1 Cartesian Product Operator ($\times$)
- 2 Rename Operator (ρ)
- 3 Join Operation ($\bowtie$)
- 4 Types of JOINS
- 5 Summary

- 1 Cartesian Product Operator ($\times$)
- 2 Rename Operator (ρ)
- 3 Join Operation ($\bowtie$)
- 4 Types of JOINS
- 5 Summary

- **Cartesian Product** is a **fundamental operator** in relational algebra.
- It **combines information from two different relations** into one.
- Also called **Cross Product**.
- On its own, it is usually not meaningful; becomes meaningful when followed by **Selection** (σ) or other operators.
- **Symbol:** $\times$
- **Notation:** $R_1 \times R_2$
- **SQL Equivalent:** `SELECT * FROM R1, R2;`

Remark

The result of $R_1 \times R_2$ will contain **all possible pairs** of tuples from R_1 and R_2 .

Important

The size of the result = *number of tuples in R_1* $\times$ *number of tuples in R_2* . This can be very large!

Relation 1: Student

Roll_No	Name
1	A
2	B

Relation 2: Course

C_ID	Course
101	DBMS
102	OS

Result: Student $\times$ Course

Roll_No	Name	C_ID	Course
1	A	101	DBMS
1	A	102	OS
2	B	101	DBMS
2	B	102	OS

Examples

This operation becomes meaningful when followed by **Selection (σ)** to filter rows, which leads to **Join operations**.

Relation R1

A	B
α	1
β	2

Relation R2

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

Result: $R1 \times R2$

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

Remark

The result contains **all possible pairings** of tuples from $R1$ and $R2$. If $|R1| = m$ and $|R2| = n$, then $|R1 \times R2| = m \times n$.

- If *relation R1 and R2 have a & b attributes* respectively, then the **resulting relation** will have **$a + b$ attributes** from both the input relations.
- If *relation R1 and R2 have $n1$ & $n2$ tuples* respectively, then the **resulting relation** will have **$n1 \times n2$ tuples**, combining each possible pair of tuples from both the relations.

	R1	R2	$R1 \times R2$
Attributes	a	b	$(a + b)$
Tuples	$n1$	$n2$	$(n1 \times n2)$

Important Note

If both input relations have **some attributes with the same name**, rename them using the format:
relation_name.attribute_name.

Examples

Example: If both $R1$ and $R2$ contain an attribute ID, then rename them as $R1.ID$ and $R2.ID$.

- If both input relations have **some attribute with the same name**, rename that attribute using the format **relation_name.attribute_name**.

R1

A	B
α	1
β	2

$\Rightarrow$

$R1 \times R2$

A	R1.B	R2.B	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

R2

B	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

- Can build expressions using **multiple operations**.
- Example:** $\sigma_{A=C}(R1 \times R2)$

Result: After Selection

A	B	C	D	E
α	1	α	10	a
β	2	β	10	a
β	2	β	20	b

Cartesian Product ($R1 \times R2$)

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

Examples

Here, only tuples where **A = C** are selected.

Key Point

The Cartesian product is often not meaningful by itself. It becomes useful when **combined with selection and projection**.

- **Example:** $\Pi_A(\sigma_{D=20}(R1 \times R2))$
- Steps:
 - 1 Perform **Cartesian Product** of $R1$ and $R2$.
 - 2 Apply **Selection**: choose tuples where $D = 20$.
 - 3 Apply **Projection**: keep only attribute A .

Result: Projection on A

A
α
β

Examples

Authors of attribute A are extracted after filtering on $D = 20$.

Cartesian Product ($R1 \times R2$)

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

Key Point

Relational algebra allows **nesting of operations**. Here, σ (Selection) and Π (Projection) are applied after the Cartesian Product.

- 1 Cartesian Product Operator ($\times$)
- 2 Rename Operator (ρ)
- 3 Join Operation ($\bowtie$)
- 4 Types of JOINS
- 5 Summary

- The results of relational algebra are also relations but without any name.
- The **RENAME** operator is used to rename the output of a relation.
- Sometimes it is simple and suitable to break a complicated sequence of operations and rename it as a relation with different names. Reasons to rename a relation can be many:
 - Save the result of a relational algebra expression as a relation so that we can use it later.
 - Avoid confusion when joining (Cartesian product) a relation with itself, by renaming one of the tables.

Symbol

Rho (ρ)

Examples

Notation 1: $\rho_X(E)$

E is the result of expression or sequence of operations which is saved with the name X .

- **SQL:** Use the **AS** keyword in the **FROM** clause
 - Example: Students AS Stud1 (renames Students to Stud1)
 - `SELECT * FROM Students AS Stud1;`
- **SQL:** Use the **AS** keyword in the **SELECT** clause to rename attributes
 - Example: RollNo AS ID (renames RollNo to ID)
 - `SELECT RollNo AS ID, SName FROM Students;`

Important

Renaming helps improve query readability and resolves ambiguity.

Notation 2

$\rho_X(A_1, A_2, \dots, A_n)(E)$: Returns the result of expression E under the name X , with the attributes renamed to $A_1, A_2, \dots, A_n$.

Notation 3

$\rho(A_1, A_2, \dots, A_n)(E)$: Returns the result of expression E with the attributes renamed to $A_1, A_2, \dots, A_n$ (relation name unchanged).

Why it matters

Renaming helps chain complex expressions, improves readability, and avoids attribute ambiguity in multi-relation expressions.

- When doing $R \times R$ (same table twice), **rename one copy** to avoid ambiguity.

Relation R

A	B
α	1
β	2

Ambiguous: $R \times R$

R.A	R.B	R.A	R.B
α	1	α	1
α	1	β	2
β	2	α	1
β	2	β	2

Correct: $R \times \rho_S(R)$

R.A	R.B	S.A	S.B
α	1	α	1
α	1	β	2
β	2	α	1
β	2	β	2

Examples

Expression: $R \times \rho_S(R)$

Example 1: Rename Result Relation and Attributes

Task: Find the female students from Student and rename the result relation to FemaleStudent with attributes SNo, Name.

Examples

Relational Algebra: $\rho_{\text{FemaleStudent}}(\text{SNo, Name}) \left(\pi_{\text{RollNo, SName}} \left(\sigma_{\text{Gender}='F'}(\text{Student}) \right) \right)$

Student	RollNo	SName	Gender
	101	Asha	F
	102	Ritesh	M
	103	Meera	F
	104	Kiran	M
	105	Leena	F

$\Rightarrow$

FemaleStudent	
SNo	Name
101	Asha
103	Meera
105	Leena

- **Example-2:** Rename the attributes Name, Age of table Person to N, A.

$$\rho_{(N,A)}(\text{Person})$$

- **Example-3:** Rename the table Project to Work and its attributes to P, Q, R.

$$\rho_{\text{Work}(P,Q,R)}(\text{Project})$$

- **Example-4:** Rename the first attribute of table Student with attributes A, B, C to P.

$$\rho_{(P,B,C)}(\text{Student})$$

- **Example-5:** Rename the table Loan to L.

$$\rho_L(\text{Loan})$$

Note

All three notations are widely used in textbooks and exam questions. Be comfortable reading and writing each.

- 1 Cartesian Product Operator ($\times$)
- 2 Rename Operator (ρ)
- 3 Join Operation ($\bowtie$)
- 4 Types of JOINS
- 5 Summary

Definition

Join is an **Additional / Derived Operator** which simplifies queries but does not add new power to relational algebra.

- A **Join** is a **combination of Cartesian Product followed by Selection**.
- Mathematically:

$$A \bowtie_C B = \sigma_C(A \times B)$$

- It pairs tuples from different relations **only if the join condition is satisfied**.

Key Idea

Join = **Cartesian Product + Selection**

Symbol : $\bowtie$

Advantages

- Reduces large Cartesian product results to **meaningful combinations**.
- Allows combining data across multiple relations (like SQL JOIN).
- Essential in queries involving conditions across tables.

Example

SQL equivalent: `SELECT * FROM Student S, Department D WHERE S.DeptId = D.DeptId;`

Relational Algebra: $Student \bowtie_{Student.DeptId=Dept.DeptId} Department$

Joins ($\bowtie$)

- **Combination of tuples** that satisfy the filtering/matching conditions.
- Produces **fewer tuples** compared to Cartesian Product.
- More efficient for meaningful queries.

Natural Join: $R \bowtie S$

A	B	C
10	x	50
20	y	60

Cartesian Product ($\times$)

- Produces **all possible combinations** of tuples.
- Generates a **huge number of tuples**, often costly to manage.

$R \times S$

A	R.B	S.B	C
10	x	x	50
10	x	y	60
20	y	x	50
20	y	y	60

- 1 Cartesian Product Operator ($\times$)
- 2 Rename Operator (ρ)
- 3 Join Operation ($\bowtie$)
- 4 Types of JOINS**
- 5 Summary

1. Inner Join

- Contains only those tuples that **satisfy the matching condition**.
- Variants:
 - Theta (θ) Join
 - Equi Join
 - Natural Join

2. Outer Join

- **Extension of Join.**
- Contains matching tuples + tuples that do not satisfy the condition.
- Includes all rows from one or both relations.
- Variants:
 - Left Outer Join
 - Right Outer Join
 - Full Outer Join

Examples

Inner Join = strict matching only. Outer Join = matching + unmatched rows.

1. Inner Join

- Contains only those tuples that **satisfy the matching condition**.
- **Theta (θ) / Conditional Join**
 - $A \bowtie_{\theta} B$
 - Uses all comparison operators:
 $<, >, \leq, \geq, =, \neq$
- **Equi Join**
 - A special case of θ -join
 - Uses only equality ($=$) as comparison
- **Natural Join**
 - $A \bowtie B$
 - Based on **common attributes** in both relations
 - Does not require any explicit comparison operator

2. Outer Joins

- Extension of join: Contains tuples satisfying the matching condition Plus tuples that do **not** satisfy it
- **Left Outer Join**
 - All tuples from left relation always appear
- **Right Outer Join**
 - All tuples from right relation always appear
- **Full Outer Join**
 - All tuples from both relations appear, matched or not

Input Relation S1

sid	name	rating	age
12	alice	6	42.0
27	berto	8	33.5
44	rina	9	29.0

sid	bid	day
12	101	2022-05-04
44	103	2022-06-10

Join Condition (Theta)

We pair tuples where $S1.sid < R1.sid$.

$$S1 \bowtie_{S1.sid < R1.sid} R1 \equiv \sigma_{S1.sid < R1.sid}(S1 \times R1)$$

Output

S1.sid	name	rating	age	R1.sid	bid	day
12	alice	6	42.0	44	103	2022-06-10
27	berto	8	33.5	44	103	2022-06-10

Input Relation *S1*

sid	name	rating	age
12	alice	6	42.0
27	berto	8	33.5
44	rina	9	29.0

Input Relation *R1*

sid	bid	day
12	101	2022-05-04
44	103	2022-06-10

Join Condition (Equi)

Match when *S1.sid = R1.sid*.

$$S1 \bowtie_{S1.sid=R1.sid} R1 \equiv \sigma_{S1.sid=R1.sid}(S1 \times R1)$$

Result

S1.sid	name	rating	age	R1.sid	bid	day
12	alice	6	42.0	12	101	2022-05-04
44	rina	9	29.0	44	103	2022-06-10

Example 2: Equi Join (Student–Subjects)

	SID	Name	Std
Student	201	Rohan	9
	202	Mira	10

Join Condition

Student $\bowtie$ *Student.Std=Subjects.Class* Subjects

	Class	Subject
Subjects	9	Algebra
	9	Physics
	10	Biology
	10	Chemistry

	SID	Name	Std	Class	Subject
Output	201	Rohan	9	9	Algebra
	201	Rohan	9	9	Physics
	202	Mira	10	10	Biology
	202	Mira	10	10	Chemistry

Observation

With multiple subjects per class, an **equi join duplicates** the student row once for each matching subject.

- **Natural join** can only be performed *if there is at least one common attribute* (column) that exists between two relations.
- The attributes must have the **same name and domain**.
- Natural join does not use any comparison operator.
- *It is same as equi join which occurs implicitly* by comparing all the common attributes (columns) in both relations, **but** difference is that **in Natural Join the common attributes appear only once**. The resulting schema will change.

Notation: $A \bowtie B$

Result

The result of the natural join is the **set of all combinations of tuples** in two relations A and B that are **equal on their common attribute names**.

Note

- The **Natural Join** of two relations can be *obtained* by applying a **Projection operation** to **Equi join** of two relations.
- In terms of basic operators:

Natural Join = Cartesian product + Selection + Projection

Important Point

- **Natural Join ($\bowtie$)** is by default an **Inner Join** because tuples which do not satisfy the condition of join **do not appear in the result set**.
- **Natural Join** is very important in relational algebra.

- $r = (A, B, C, D)$
- $s = (B, D, E)$
- **Resulting schema** of $r \bowtie s = (A, B, C, D, E)$

Definition

$r \bowtie s$ is defined as:

$$\Pi_{A,r.B,C,r.D,E}(\sigma_{r.B=s.B \wedge r.D=s.D}(r \times s))$$

Relation r

A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	B

Relation s

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ϵ

$r \bowtie s$

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
β	2	β	b	δ

Courses

CID	Course	Dept
CS01	Database	CS
ME01	Mechanics	ME
EE01	Electronics	EE

HOD

Dept	Head
CS	Rohan
ME	Sara
EE	Jiya

Natural Join Result

Courses $\bowtie$ HOD

CID	Course	Dept	Head
CS01	Database	CS	Rohan
ME01	Mechanics	ME	Sara
EE01	Electronics	EE	Jiya

Equivalent to: $\Pi_{CID, Course, Courses.Dept, Head} (\sigma_{Courses.Dept=HOD.Dept}(Courses \times HOD))$

S1

sid	name	rating	age
22	Dustin	7	45.0
31	Lubber	8	55.0
58	Rusty	10	35.0

R1

sid	bid	day
22	101	10/10/96
58	103	11/12/96

Natural Join Result: $S1 \bowtie R1$

sid	name	rating	age	bid	day
22	Dustin	7	45.0	101	10/10/96
58	Rusty	10	35.0	103	11/12/96

Examples

Equivalent to:

$$\Pi_{S1.sid, name, rating, age, bid, day} (\sigma_{S1.sid=R1.sid} (S1 \times R1))$$

Concept

An **Inner join** includes only those tuples with matching attributes and the rest are discarded in the resulting relation. Therefore, we need to use **Outer Joins** to include all the rest of the tuples from the participating relations in the resulting relation.

Definition

The **Outer Join** operation is an **extension of the join operation** that **avoids loss of information**.

- **Outer Join** contains matching tuples that satisfy the matching condition, along with some or all tuples that do not satisfy the matching condition.
 - It is based on both matched or unmatched tuples.
 - It contains all rows from either one or both relations.
- It **uses NULL values**.
 - NULL signifies that the value is unknown or does not exist.

1. Left Outer Join ($R1 \bowtie R2$)

Concept

When applying **join** on two relations R1 and R2, some tuples of R1 or R2 do not appear in the result set if they do not satisfy the join condition. **But...**

Definition

In **Left Outer Join**, **all the tuples from the Left relation R1 are included** in the resulting relation. The tuples of R1 which do not satisfy the join condition will have values as NULL for attributes of R2.

In short:

- All records from the **left** table
- Only matching records from the right table

Symbol: $\bowtie$

Notation: $R1 \bowtie R2$

Examples (Note)

Left Outer Join ensures no data is lost from the left relation.

Courses

CID	Course
100	Database
101	Mechanics
102	Electronics

HOD

CID	Name
100	Rohan
102	Sara
104	Jiya

Operation: **Courses** ⋈ **HOD**

Result

CID	Course	Name
100	Database	Rohan
101	Mechanics	NULL
102	Electronics	Sara

Examples

The unmatched tuple from **Courses** (CID=101) is still included, with **NULL** for the missing HOD.

- In **Right outer join**, all the tuples from the right relation $R2$ are included in the resulting relation.
- The tuples of $R2$ which do not satisfy join condition will have values as **NULL** for attributes of $R1$.

In short

- All records from **right** table.
- Only matching records from **left** table.

Symbol: $\bowtie$ (outer join symbol)

Notation: $R1 \bowtie R2$

Courses

CID	Course
100	Database
101	Mechanics
102	Electronics

HOD

CID	Name
100	Rohan
102	Sara
104	Jiya

Courses $\times$ HOD

CID	Course	Name
100	Database	Rohan
102	Electronics	Sara
104	NULL	Jiya

3. Full Outer Join ($R1 \bowtie^{full} R2$)

- In **Full outer join**, all the tuples from both Left relation $R1$ and Right relation $R2$ are included in the resulting relation.
- The tuples of both relations $R1$ and $R2$ which do not satisfy join condition, their respective unmatched attributes are made **NULL**.

In short:

- All records from **all** tables are preserved.
- Unmatched attributes are filled with **NULL**.
- **Symbol:** $\bowtie^{full}$
- **Notation:** $R1 \bowtie^{full} R2$

Relations:

Courses

CID	Course
100	Database
101	Mechanics
102	Electronics

HOD

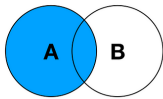
CID	Name
100	Rohan
102	Sara
104	Jiya

Result: Courses $\bowtie^{full}$ HOD

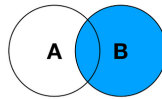
CID	Course	Name
100	Database	Rohan
101	Mechanics	NULL
102	Electronics	Sara
104	NULL	Jiya

- 1 Cartesian Product Operator ($\times$)
- 2 Rename Operator (ρ)
- 3 Join Operation ($\bowtie$)
- 4 Types of JOINS
- 5 Summary

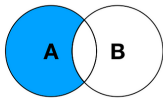
SQL JOINS



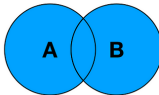
LEFT JOIN



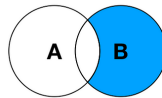
RIGHT JOIN



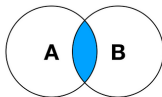
LEFT JOIN EXCLUDING
INNER JOIN



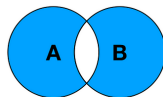
FULL OUTER JOIN



RIGHT JOIN EXCLUDING
INNER JOIN



INNER JOIN



FULL OUTER JOIN EXCLUDING
INNER JOIN

Cartesian Product ($R \times S$)

- Combines every tuple of relation R with every tuple of relation S . Produces **all possible combinations**.
- Schema = attributes of R + attributes of S . Can create **large result sets**.

Examples

Use Case: Intermediate step in **Join** and other relational algebra operations.

Rename Operator (ρ)

- Used to **rename relations or attributes**. Useful when two relations have attributes with the **same name**.
- Example: $\rho_{R1.A, R2.A}(R1 \times R2)$

Join ($\bowtie$)

- **Derived operator:** Combination of **Cartesian Product + Selection**.
- Matches tuples from two relations based on a **condition**.
- Reduces size compared to Cartesian Product.

Examples

Types of Joins:

- **Inner Joins:** Theta (θ), Equi, Natural
- **Outer Joins:** Left, Right, Full (preserve unmatched tuples with **NULLs**)

Key Point

Joins are essential for combining related data across multiple relations.

Thank You!



Relational Algebra - 3

DBMS Lecture Series

Unit-3

Pavinder Yadav

Assistant Professor
School of Computer Science
University of Petroleum and Energy Studies
Dehradun



- 1 Division Operator ($\div$, /)
- 2 Assignment Operator ($\leftarrow$)
- 3 Extended Relational Algebra
- 4 NULL Values in aggregate expression

- 1 Division Operator ($\div$, /)
- 2 Assignment Operator ($\leftarrow$)
- 3 Extended Relational Algebra
- 4 NULL Values in aggregate expression

The **Division operator** is a **Derived Operator**, not supported as a primitive operator.

Usage

Suited to queries that include keywords like "**all**" or "**every**" (e.g., *at all*, *for all*, *in all*, *at every*, *for every*, *in every*).

Examples:

- Find the person that has account in **all** the banks of a city.
- Find sailors who have reserved **all** boats.
- Find employees who work on **all** projects of the company.
- Find students who have registered for **every** course.

In all such queries, the part after "**all**" or "**every**" defines a *set* of requirements. The **final result** contains only those tuples that satisfy **all requirements**.

Notation: $A \div B$ or A/B , where A and B are two relations.

Relational Algebra Example

Sailors $\div$ Boats

Find sailors who have reserved
all boats.

Examples

SQL Equivalent:

```
SELECT S.sname
FROM Sailors S
WHERE NOT EXISTS (
    SELECT B.bid
    FROM Boats B
    WHERE NOT EXISTS (
        SELECT R.bid
        FROM Reserves R
        WHERE R.sid = S.sid AND R.bid = B.
            bid
    )
);
```

Relational Algebra

$$A \div B$$

- Works with two relations.
- Keeps tuples in A associated with **all tuples** in B .

SQL Equivalent

- Often implemented using **NOT EXISTS**.
- Alternative: use **GROUP BY** and **HAVING COUNT**.

Examples

Find sailors who reserved all boats:

```
SELECT R.sid
FROM Reserves R
GROUP BY R.sid
HAVING COUNT(DISTINCT R.bid) =
        (SELECT COUNT(*) FROM Boats);
```


Can be applied if and only if

- Attributes of B is a **proper subset** of attributes of A .
- The relation returned by division operator will have

Attributes = (All attributes of A – All attributes of B)

- The result will return tuples from A which are associated to **every tuple of B** .

Relation $A(x,y)$:

x	y
a	1
b	2
a	2
d	4

Relation $B(y)$:

y
1
2

Result $(A \div B)$:

x
a

Observation

Only a is related with **all values** of B (1 and 2).

Examples

Find x values from A associated with all y in B :

```
SELECT A.x
FROM A
WHERE NOT EXISTS (
    SELECT B.y
    FROM B
    WHERE NOT EXISTS (
        SELECT A1.y
        FROM A A1
        WHERE A1.x = A.x AND A1.y = B.y
    )
);
```

Examples

Find x values from A related with all y in B :

```
SELECT A.x
FROM A
GROUP BY A.x
HAVING COUNT(DISTINCT A.y) = (SELECT COUNT(*) FROM B);
```

Example 2: Division Operator Working

Relation A(sno, pno)

sno	pno
S1	P1
S1	P2
S1	P3
S1	P4
S2	P1
S2	P2
S3	P2
S4	P2
S4	P4

B1

pno
P2

B2

pno
P2
P4

B3

pno
P1
P2
P4

Results:

$$A \div B1 =$$

sno
S1
S2
S3
S4

$$A \div B2 =$$

sno
S1
S4

$$A \div B3 =$$

sno
S1

Key Points

- Division is a **derived operator** (or additional operator).
- It can be expressed in terms of **Cross Product**, **Set Difference**, and **Projection**.

For $A \div B$, compute all x values that are not **disqualified** by some y in B .

- An x value is **disqualified** if, by attaching a y from B , we obtain an xy tuple not in A .

In Relational Algebra

Disqualified x values:

$$\pi_x((\pi_x(A) \times B) - A)$$

So,

$$A \div B = \pi_x(A) - \pi_x((\pi_x(A) \times B) - A)$$

Relation A(A,B,C,D,E)

A	B	C	D	E
α	a	α	a	1
α	a	γ	a	1
α	a	γ	b	1
β	a	γ	a	1
β	a	γ	b	3
γ	a	γ	a	1
γ	a	γ	b	1
γ	a	β	b	1

Relation B(D,E)

D	E
a	1
b	1

Result ($A \div B$):

A	B	C
α	a	γ
γ	a	γ

Observation

Only those (A, B, C) combinations remain where tuples match for **all** (D,E) values in B.

- 1 Division Operator ($\div$, /)
- 2 Assignment Operator ($\leftarrow$)
- 3 Extended Relational Algebra
- 4 NULL Values in aggregate expression

Definition

The **assignment operation** ($\leftarrow$) provides a convenient way to express complex queries.

Examples

It writes a query as a sequential program consisting of:

- A series of **assignments**.
- Followed by an expression whose value is displayed as the **result**.

Note: Assignment must always be made to a **temporary relation variable**.

Relational Algebra

$$Temp \leftarrow \pi_{sname}(Sailors)$$

Then use *Temp* in further operations.

SQL Equivalent:

```
-- Using WITH clause for assignment
WITH Temp AS (
    SELECT sname
    FROM Sailors
)
SELECT *
FROM Temp;
```

Alternative:

```
-- Using a temporary table
CREATE TEMP TABLE Temp AS
SELECT sname
FROM Sailors;

SELECT * FROM Temp;
```

Relational Algebra Expression

$$A \div B = \pi_x(A) - \pi_x((\pi_x(A) \times B) - A)$$

Step-by-step using assignment:

$$temp1 \leftarrow \pi_x(A)$$

$$temp2 \leftarrow \pi_x((temp1 \times B) - A)$$

$$result \leftarrow temp1 - temp2$$

Note

- The result on the right of $\leftarrow$ is assigned to the relation variable on the left.
- These temporary variables may be used in subsequent expressions.

- 1 Division Operator ($\div$, /)
- 2 Assignment Operator ($\leftarrow$)
- 3 Extended Relational Algebra**
- 4 NULL Values in aggregate expression

Definition

Extended Relational Algebra increases power over **basic relational algebra**.

- Generalized Projection
- Aggregate Functions
- Outer Join

Definition

- Normal *projection* only projects the **columns**.
- **Generalized Projection** allows **arithmetic operations** on those projected columns.

Generalized Projection extends the **projection operation** by allowing **arithmetic functions** to be used in the *projection list*.

$$\pi_{F_1, F_2, \dots, F_n}(E)$$

- E is any relational-algebra expression.
- $F_1, F_2, \dots, F_n$ are arithmetic expressions involving **constants and attributes** in the schema of E .

Note

- Useful for calculations such as **salary + bonus**, **price * quantity**, etc.
- Reduces the need to compute derived values separately.

Example 1: Generalized Projection

Relation $r(A,B,C)$

A	B	C
a	1	5
a	2	5
b	3	8
b	4	10

Result of

$$\pi_{A, C-B}(r)$$

A	$C - B$
a	4
a	3
b	5
b	6

Result of

$$\pi_{A, 2C-B}(r)$$

A	$2C - B$
a	9
a	8
b	13
b	16

Observation

Here, the projection applies an **arithmetic operation** $C - B$, and $2C - B$ instead of projecting only attributes. This is the power of **generalized projection**.

Given Relation

$credit_info(customer_name, limit, credit_balance) \quad \pi_{customer_name, limit - credit_balance}(credit_info)$

Query: Find how much more each person can spend.

Output:

<i>customer_name</i>	<i>limit</i>	<i>credit_balance</i>
A	5000	2000
B	6000	4000
C	10000	6000

<i>customer_name</i>	<i>limit - credit_balance</i>
A	3000
B	2000
C	4000


```
SELECT customer_name ,  
       (limit - credit_balance) AS remaining_amount  
FROM credit_info;
```

Observation

This query directly applies arithmetic operations in **projection**, just like generalized projection in relational algebra.

Definition

An **aggregation function** takes a *collection of values* and returns a **single value** as a result.

Common aggregate functions:

- **avg**: average value
- **min**: minimum value
- **max**: maximum value
- **sum**: sum of values
- **count**: number of values

Notes

- These can be applied on the **entire relation** or on **groups of tuples**.
- NULL values are ignored (except in **count**).

$$G_1, G_2, \dots, G_n \ g_{F_1(A_1), F_2(A_2), \dots, F_n(A_n)}(E)$$

Explanation

- E : any relational-algebra expression.
- $G_1, G_2, \dots, G_n$: attributes on which to **group** (can be empty).
- Each F_i : an **aggregate function** (e.g., SUM, AVG).
- Each A_i : an **attribute name**.

Examples

Example: Find the average salary for each department.

$$dept_id \ g_{avg(salary)}(Employee)$$

```
-- Average salary of all employees
SELECT AVG(salary)
FROM Employee;

-- Maximum salary in each department
SELECT dept_id, MAX(salary)
FROM Employee
GROUP BY dept_id;

-- Count of employees in each department
SELECT dept_id, COUNT(*)
FROM Employee
GROUP BY dept_id;
```

Observation

Relational Algebra aggregate operations map directly to SQL **GROUP BY** and **aggregate**

Aggregate Results:

Relation $r(A,B,C)$

A	B	C
a	a	5
a	b	5
b	c	2
b	d	3

$\Rightarrow$

$g_{sum(C)}(r)$

$sum(C)$
20

$A g_{sum(C)}(r)$

A	$sum(C)$
a	10
b	5

Observation

- Without grouping $\rightarrow$ total sum of C values.
- With grouping by $A \rightarrow$ sum of C values per group of A .

```
-- Total sum of column C
SELECT SUM(C) AS total_sum
FROM r;

-- Sum of C grouped by A
SELECT A, SUM(C) AS sum_c
FROM r
GROUP BY A;
```

Examples

Results:

- Total sum = 20
- Grouped by A: $a \rightarrow 10$, $b \rightarrow 5$

Relation: **account(branch_name, account_number, balance)**

Aggregate Result:

$branch_name \ g_{sum(balance)}(account)$

<i>branch_name</i>	<i>account_number</i>	<i>balance</i>
Perryridge	A – 102	400
Perryridge	A – 201	900
Brighton	A – 217	750
Brighton	A – 215	750
Redwood	A – 222	700

⇒

<i>branch_name</i>	<i>sum(balance)</i>
Perryridge	1300
Brighton	1500
Redwood	700

Observation

Grouping by **branch_name** and applying **SUM(balance)** gives the total balance per branch.

```
SELECT branch_name, SUM(balance) AS total_balance  
FROM account  
GROUP BY branch_name;
```

Examples

Result:

- Perryridge → 1300
- Brighton → 1500
- Redwood → 700

Issue

The **result of aggregation** does not have a name.

Examples

- We can use the **rename operation** to give it a name.
- For convenience, renaming can be written as part of the aggregate expression using the keyword **AS**.

branch_name $g_{sum(balance)}$ as **sum_balance**(*account*)

Example

Instead of just showing $sum(balance)$, we rename it as **sum_balance**.

- 1 Division Operator ($\div$, /)
- 2 Assignment Operator ($\leftarrow$)
- 3 Extended Relational Algebra
- 4 NULL Values in aggregate expression

Definition

It is possible for **tuples** to have a **NULL value**, denoted by NULL, for some of their attributes.

Examples

NULL signifies:

- An **unknown value** or missing information.
- A value that **does not exist**.

Example 1:

<i>Emp_Id</i>	<i>Name</i>	<i>Phone_No</i>	<i>Passport_No</i>
001	Rahul	7777777777	11111111
002	Anil	8888888888	NULL

Example 2:

<i>First_Name</i>	<i>Middle_Name</i>	<i>Last_Name</i>
Ranjit	Singh	Thakur
Amit	NULL	Chopra

Arithmetic with NULL The result of any **arithmetic expression** (+, -, *, /) involving NULL must return a NULL.

Examples: $5 + \text{NULL} = \text{NULL}$, $\text{NULL} * 5 = \text{NULL}$

Aggregate Functions

Aggregate functions simply **ignore NULL values** (as in SQL).

- AVG, MIN, MAX, SUM → ignore NULL.
- COUNT → considers NULL handling differently (COUNT(*) counts NULLs, but COUNT(column) ignores them).

Grouping and Duplication

For duplication, elimination, and grouping, NULL is *treated like any other value*, and two NULLs are assumed to be the same (as in SQL).

Comparisons

Comparisons ($<$, $\leq$, $>$, $\geq$, $=$, $\neq$) with NULL values evaluate to the special value **unknown** (as in SQL).

Example: $5 = \text{NULL}$, $\text{NULL} > 5$, $5 > \text{NULL}$, $\text{NULL} = \text{NULL} \Rightarrow \text{unknown}$

Boolean Expressions

Comparisons in Boolean expressions involving **AND**, **OR**, **NOT** use **three-valued logic**:

$$\{\text{true}(1), \text{false}(0), \text{unknown}\}$$

Note

The result of a SELECT predicate is treated as **false** if it evaluates to unknown.

Examples

Three-Valued Logic:

OR:

$(unknown \text{ or } true) = true$

$(unknown \text{ or } false) = unknown$

$(unknown \text{ or } unknown) = unknown$

AND:

$(true \text{ and } unknown) = unknown$

$(false \text{ and } unknown) = false$

$(unknown \text{ and } unknown) = unknown$

NOT:

$(not \text{ unknown}) = unknown$

Division Operator

- A **derived operator** used for queries with **ALL / EVERY**.
- Expressed using **Cross Product, Projection, and Set Difference**.
- Example: Find employees who work on **all projects**.

Assignment Operator

- Allows breaking complex queries into steps.
- Temporary relation variables store intermediate results.
- SQL equivalent: **WITH clause** or temporary tables.

Extended Relational Algebra

- **Generalized Projection**: allows arithmetic expressions.
- **Aggregate Functions**: avg, sum, min, max, count.
- **Outer Join**: includes unmatched tuples.

Aggregate Functions

- Operate on collections of values → return a single value.
- Can be grouped by attributes (**GROUP BY**).
- NULL values are ignored (except in **COUNT(*)**).

NULL Values

- Represent **unknown or missing values**.
- Arithmetic with NULL → result is **NULL**.
- Comparisons with NULL → result is **unknown**.
- SQL uses **three-valued logic**: true, false, unknown.

Key Takeaway

Relational Algebra concepts map closely to SQL constructs, providing a strong foundation for query optimization and database design.

Thank You!



Relational Algebra - 4

DBMS Lecture Series

Unit-3

Pavinder Yadav

Assistant Professor
School of Computer Science
University of Petroleum and Energy Studies
Dehradun



- 1 Modification of the Database: *Deletion, Insertion, Updating*
- 2 Views in DBMS
- 3 Summary

- 1 Modification of the Database: *Deletion, Insertion, Updating*
- 2 Views in DBMS
- 3 Summary

Definition

The **content of the database** may be **modified** using the following operations:

- **Deletion**
- **Insertion**
- **Updating**

Note

All these operations are expressed using the **assignment operator** ($\leftarrow$).

Examples

Example:

$$r_{new} \leftarrow \text{operations on } (r_{old})$$

Definition

A **delete** request is expressed similarly to a query, except instead of displaying tuples to the user, the **selected tuples are removed from the database**.

Examples

- In **Deletion**, **tuples are deleted from the relation**.
- Can delete only **whole tuples**; cannot delete values of individual attributes.

Relational Algebra Expression: $r \leftarrow r - E$, where r is a relation and E is a relational algebra expression.

Note

Deletion is a **destructive operation**: once deleted, tuples are removed from the relation unless restored.

```
-- Delete all employees in  
    department 5  
DELETE FROM Employee  
WHERE dept_id = 5;  
  
-- Delete all accounts with  
    balance < 500  
DELETE FROM Account  
WHERE balance < 500;
```

Examples

Relational Algebra:

$$Employee \leftarrow Employee - \sigma_{dept_id=5}(Employee)$$
$$Account \leftarrow Account - \sigma_{balance < 500}(Account)$$

Observation

Relational Algebra expresses deletion as *removing tuples*, while **SQL** uses the DELETE FROM ... WHERE command.

Relation: CSE_Students

Roll_No	Name	Backlogs
201	Aditi	0
202	Rahul	2
203	Sneha	1
204	Karan	3

Operation: Delete all students with Backlogs = 2

$$CSE_Students \leftarrow CSE_Students - \sigma_{Backlogs=2}(CSE_Students)$$

Result:

Roll_No	Name	Backlogs
201	Aditi	0
203	Sneha	1
204	Karan	3

SQL Equivalent

```
DELETE FROM CSE_Students
WHERE Backlogs = 2;
```

Relation: CSE_Students

Roll_No	Name	Backlogs
201	Aditi	0
202	Rahul	2
203	Sneha	1
204	Karan	3

Operation: Delete student with tuple
(202, Rahul, 2)

$$CSE_Students \leftarrow CSE_Students - \{(202, Rahul, 2)\}$$

Result:

Roll_No	Name	Backlogs
201	Aditi	0
203	Sneha	1
204	Karan	3

SQL Equivalent

```
DELETE FROM CSE_Students
WHERE Roll_No = 202
AND Name = 'Rahul'
AND Backlogs = 2;
```

- Delete all account records in the Perryridge branch.

$$account \leftarrow account - \sigma_{branch_name="Perryridge"}(account)$$

- Delete all loan records with amount in the range of 0 to 50.

$$loan \leftarrow loan - \sigma_{amount \geq 0 \wedge amount \leq 50}(loan)$$

- Delete all accounts at branches located in Dehradun.

$$r_4 \leftarrow \sigma_{branch_city="Dehradun"}(account \bowtie branch)$$

$$r_5 \leftarrow \Pi_{account_number, branch_name, balance}(r_4)$$

$$r_6 \leftarrow \Pi_{customer_name, account_number}(r_5 \bowtie depositor)$$

$$account \leftarrow account - r_5, \quad depositor \leftarrow depositor - r_6$$

```
-- Delete all account records in the Perryridge branch
DELETE FROM Account
WHERE branch_name = 'Perryridge';

-- Delete all loan records with amount between 0 and 50
DELETE FROM Loan
WHERE amount BETWEEN 0 AND 50;

-- Delete all accounts at branches located in Dehradun
DELETE FROM Account
WHERE branch_name IN (
    SELECT branch_name FROM Branch WHERE branch_city = 'Dehradun'
);
```

- **Similar to deletion**, but uses $\cup$ (**Union**) **operator** instead of $-$ operator.
- **In Insertion**, tuples are **added** to the relation.
- To **insert** data into a relation, we either:
 - Specify a **tuple to be inserted**, or
 - Write a **query** whose result is a set of tuples to be inserted.
- An insertion is expressed in Relational Algebra as: $r \leftarrow r \cup E$ where r is a relation and E is a relational algebra expression.
- Insertion of a single tuple is expressed by letting E be a constant relation containing one tuple.

SQL Equivalent

```
INSERT INTO CSE_Students (Roll_No, Name, Backlogs)
VALUES (205, 'Mehul', 0);
```

Operation: Insert tuple (204, *Karan*, 0)

Relation: CSE_Students

<i>Roll_No</i>	<i>Name</i>	<i>Backlogs</i>
201	<i>Aditi</i>	0
202	<i>Rahul</i>	2
203	<i>Sneha</i>	1

$CSE_Students \leftarrow CSE_Students \cup \{(204, Karan, 0)\}$

Result:

<i>Roll_No</i>	<i>Name</i>	<i>Backlogs</i>
201	<i>Aditi</i>	0
202	<i>Rahul</i>	2
203	<i>Sneha</i>	1
204	<i>Karan</i>	0

- Insert information that **Smith** has \$1200 in account A-973 at the Perryridge branch:

$$account \leftarrow account \cup \{(A-973, Perryridge, 1200)\}$$

$$depositor \leftarrow depositor \cup \{(Smith, A-973)\}$$

- We may want to insert tuples on the basis of the **result of a query**.
- Example: Provide as a gift for all loan customers in the Perryridge branch, a \$200 savings account. Let the loan number serve as the account number.

$$r_1 \leftarrow \sigma_{branch_name=Perryridge}(borrower \bowtie loan)$$

$$account \leftarrow account \cup \pi_{loan_number, branch_name, 200}(r_1)$$

$$depositor \leftarrow depositor \cup \pi_{customer_name, loan_number}(r_1)$$

SQL Equivalent

-- Single tuple insertion

```
INSERT INTO Account VALUES ('A-973','Perryridge',1200);
```

```
INSERT INTO Depositor VALUES ('Smith','A-973');
```

-- Using subquery

```
INSERT INTO Account (acc_no, branch_name, balance)
```

```
SELECT loan_no, branch_name, 200
```

```
FROM Borrower NATURAL JOIN Loan
```

```
WHERE branch_name = 'Perryridge';
```


- Suppose all **CSE 2nd Year students with CGPA > 9.0** get added to the **Dean's Merit List**.

$$r_1 \leftarrow \sigma_{CGPA > 9.0}(CSE_Students)$$

$$MeritList \leftarrow MeritList \cup \pi_{Roll_No, Name}(r_1)$$

SQL Equivalent

```
INSERT INTO MeritList (Roll_No, Name)
SELECT Roll_No, Name
FROM CSE_Students
WHERE CGPA > 9.0;
```

- **Updating** is a mechanism to **change a value in a tuple** without changing all values in the tuple.
- Uses the **generalized projection** operator to perform update.

$$r \leftarrow \Pi_{F_1, F_2, \dots, F_n}(r)$$

- Each F_i can be:
 - The **attribute** of r , or
 - An **expression** involving constants and attributes of r (to give new value).
- **Note:** Schema of the updated relation must match the original schema.

Relation r

A	B	C
1	1	10
1	2	10
2	4	20

$$r \leftarrow \Pi_{A, 2*B, C}(r)$$

A	B	C
1	2	10
1	4	10
2	8	20

$$r \leftarrow \Pi_{A, 2*B, C}(\sigma_{A=1}(r))$$

A	B	C
1	2	10
1	4	10

Task: Double the values of column B in relation r .

$$r \leftarrow \Pi_{A, 2*B, C}(r)$$

Equivalent SQL:

```
UPDATE r  
SET B = B * 2;
```

- **Example 1:** Make interest payments by increasing all balances by 5%.

$$account \leftarrow \Pi_{account_number, branch_name, balance * 1.05}(account)$$

SQL Equivalent:

```
UPDATE account
SET balance = balance * 1.05;
```

- **Example 2:** Pay all accounts with balances over \$10,000 a 6% interest and pay all others 5%.

$$account \leftarrow \Pi_{account_number, branch_name, balance * 1.06}(\sigma_{balance > 10000}(account)) \cup \Pi_{account_number, branch_name, balance * 1.05}(account)$$

SQL Equivalent:

```
UPDATE account
SET balance = balance * 1.06
WHERE balance > 10000;
```

- 1 Modification of the Database: *Deletion, Insertion, Updating*
- 2 Views in DBMS
- 3 Summary

- In some cases, it is not desirable for all users to see the **entire logical model** i.e. **all the actual relations stored in the database**.
- Consider:
“A person who needs to know a customer’s loan number but has no need to see the loan amount.”

This person should see a relation described in the relational algebra by:

$$\Pi_{customer_name, loan_number}(borrower \bowtie loan)$$

- **"Views"** are **Virtual Relations (or virtual tables)**, through which a selective portion of the data from one or more relations (or tables) can be seen.
- Views **do not contain data** of their own.
- Views **do not exist physically**.

Uses of View

- Helps in **query processing** (simplifies commands, stores complex queries).
- Restricts **access to the database**.
- Hides **data complexity**.
- **Database modifications** (insert, delete, update) on views affect the **actual relations** in the database.
- Views are stored in the **data dictionary** in the table called **USER_VIEWS**.

- A **view** is defined using the CREATE VIEW statement:

Syntax

```
CREATE VIEW v AS <query expression>
```

- Here, v represents the **view name**, and the query expression is any valid relational algebra or SQL query.

Example in SQL

```
CREATE VIEW v AS  
    SELECT column_list  
    FROM table_name [WHERE condition];
```

- Once a view is defined, it can be used as a **virtual relation**.

Usage

```
SELECT * FROM v;
```

Important

- A **view definition** is not the same as creating a new relation.
- A view saves an **expression**, not data.
- Wherever the view is used, it is replaced by the equivalent **query expression at runtime**.

Creating a view (loan-customer) consisting of all loan customers and their loan number:

Relational Algebra

```
CREATE VIEW loan_customer AS  
       $\Pi_{\text{customer-name, loan-number}}(\text{borrower} \bowtie \text{loan})$ 
```

- We can find all loan customers and their loan numbers by simply using: `loan_customer`

SQL Equivalent

```
CREATE VIEW loan_customer AS  
  SELECT customer_name, loan_number  
  FROM borrower NATURAL JOIN loan;
```

Note

Wherever view `loan_customer` is used, it is replaced by the equivalent query expression at runtime.

Example 2: High Balance Accounts

Relational Algebra:

$$\text{high-balance} \leftarrow \sigma_{\text{balance} > 10000}(\text{account})$$

SQL Equivalent: CREATE VIEW high_balance AS
SELECT account_number, branch_name, balance
FROM account
WHERE balance > 10000;

Example 3: Customer Loans in Dehradun

Relational Algebra:

$$\Pi_{\text{customer-name, loan-number}}(\sigma_{\text{branch-city}='Dehradun'}(\text{borrower} \bowtie \text{loan} \bowtie \text{branch}))$$

SQL Equivalent: CREATE VIEW dehradun_loan AS
SELECT c.customer_name, l.loan_number
FROM borrower c NATURAL JOIN loan l NATURAL JOIN branch b
WHERE b.branch_city = 'Dehradun';

- 1 Modification of the Database: *Deletion, Insertion, Updating*
- 2 Views in DBMS
- 3 Summary

- **Database Modification:** Performed using assignment operator ($\leftarrow$). Includes:
 - **Deletion** – removes tuples from a relation.
 - **Insertion** – adds new tuples into a relation.
 - **Updating** – changes attribute values in tuples.
- **Views:**
 - Virtual relations (do not store data physically).
 - Provide limited access and hide data complexity.
 - Defined using `CREATE VIEW`.

- **Deletion, Insertion, Updating** help maintain and modify database contents.
- **Views** simplify query processing and restrict sensitive data.
- Views act as **virtual tables**, based on query expressions.
- All modifications on views affect the underlying base relations.

Thank You!



Normalisation - 1

DBMS Lecture Series

Unit-3

Pavinder Yadav

Assistant Professor
School of Computer Science
University of Petroleum and Energy Studies
Dehradun



- 1 Introduction to Normalisation
- 2 First Normal Form (1NF)
- 3 Functional Dependencies
- 4 Second Normal Form (2NF)
- 5 Third Normal Form (3NF)

- 1 Introduction to Normalisation
- 2 First Normal Form (1NF)
- 3 Functional Dependencies
- 4 Second Normal Form (2NF)
- 5 Third Normal Form (3NF)

- **Normalisation** is a process in DBMS to organize data in order to reduce redundancy and improve data integrity.
- It involves decomposing large tables into smaller, well-structured tables.
- Goal: Eliminate **anomalies** and ensure data consistency.

Remark

Normalisation is applied after designing the ER model and creating relational schemas.

Idea: In the table StudentInfo, we try to store entire student + branch data.

Result: Entire branch data is repeated for every student in that branch.

Redundancy: Same data stored multiple times unnecessarily.

Disadvantages:

- ❶ Insertion, Deletion, and Modification anomalies
- ❷ Inconsistency of data
- ❸ Increased database size, slower queries

Sid	Name	Age	Brcode	Brname	HOD
1	A	18	101	CS	XYZ
2	B	19	101	CS	XYZ
3	C	18	101	CS	XYZ
4	D	21	102	EC	PQR
5	E	20	102	EC	PQR
6	F	19	103	ME	KLM

Types of Anomalies:

- **Insertion Anomaly:** Cannot insert a new branch unless a student is added.
- **Deletion Anomaly:** Deleting a student may delete branch info.
- **Updation/Modification Anomaly:** Updating branch HOD requires changes in multiple rows.

Examples

Unnormalised Table: Student_Info

S_id	Name	Age	Brancode	Brname	HOD
1	A	18	101	CS	XYZ
2	B	19	101	CS	XYZ
3	C	18	101	CS	XYZ
4	D	21	102	EC	PQR
5	E	20	102	EC	PQR
6	F	19	103	ME	KLM

Examples

After Normalisation:

Student_Info (Sid, Name, Age, Brancode)

Branch_Info (Brancode, Brname, HOD)

Key Idea

Normalisation = **Decomposition of tables** to remove redundancy and anomalies, based on **functional dependencies**.

- Avoids **data redundancy** - In DBMS, redundancy is the storage of the same data in multiple locations, leading to increased storage costs, potential inconsistencies when data isn't updated uniformly, and performance issues.
- Maintains **data integrity**
- Simplifies queries
- Ensures efficient storage

Examples

Consider a single table storing:

- Student details
- Course details
- Faculty details

This may cause duplication and anomalies.

- **Insertion Anomaly**: Difficulty in inserting new data due to missing values.
- **Updation Anomaly**: Redundant data requires multiple updates.
- **Deletion Anomaly**: Unintended loss of valuable data when deleting a record.

Important

Anomalies occur when data is stored at **row level** and **column level** without proper normalisation.

Example: Unnormalised Student Table

StudentID	Name	Dept	Course_ID	Faculty
101	Raj	CSE	DBMS, OS	Dr. Sharma
102	Priya	CSE	DBMS	Dr. Sharma
103	Amit	ME	Thermo, DBMS	Dr. Singh

- Courses stored in a single column $\Rightarrow$ violates atomicity.
- Redundancy: Faculty **Dr. Sharma** repeated for every DBMS entry.
- Difficult to add new course or faculty without student record.

Example: Unnormalised Student Table

StudentID	Name	Dept	Course_ID	Faculty
101	Raj	CSE	DBMS, OS	Dr. Sharma
102	Priya	CSE	DBMS	Dr. Sharma
103	Amit	ME	Thermo, DBMS	Dr. Singh

- Courses stored in a single column $\Rightarrow$ violates atomicity.
- Redundancy: Faculty **Dr. Sharma** repeated for every DBMS entry.
- Difficult to add new course or faculty without student record.

Problems in the Table:

- **Insertion Anomaly**: Cannot insert a new course if no student enrolled yet.
- **Updation Anomaly**: If faculty name changes, must update multiple rows.
- **Deletion Anomaly**: If student 102 is deleted, information about DBMS course may be lost.

Important

The table mixes information of **Student**, **Department**, **Courses**, and **Faculty** in one place.

Schemas

- **STUDENT**(StudentID, Name, DeptID)
- **DEPARTMENT**(DeptID, DeptName)
- **COURSE**(CourseID, CourseName, DeptID)
- **FACULTY**(FacultyID, FacultyName, DeptID)
- **ENROLLMENT**(StudentID, CourseID, FacultyID)

Examples

Now:

- No data redundancy
- Easy insertion, updation, deletion
- Ensures data consistency

- Normalisation is carried out in **stages**, known as **Normal Forms**.
- Commonly used Normal Forms:
 - 1 First Normal Form (1NF)
 - 2 Second Normal Form (2NF)
 - 3 Third Normal Form (3NF)
 - 4 Boyce-Codd Normal Form (BCNF)
- Advanced Normal Forms:
 - 1 Fourth Normal Form (4NF)
 - 2 Fifth Normal Form (5NF)

Remark

We usually stop at 3NF or BCNF in practice, as they eliminate most anomalies.

- 1 Introduction to Normalisation
- 2 First Normal Form (1NF)
- 3 Functional Dependencies
- 4 Second Normal Form (2NF)
- 5 Third Normal Form (3NF)

- A relation is in **First Normal Form (1NF)** if:
 - 1 All attributes contain only **atomic (indivisible)** values.
 - 2 There are no **repeating groups** or arrays.
- In simple words: **Every field should hold a single value only.**

Important

If a table contains multi-valued attributes (like multiple courses in one column), it is NOT in 1NF.

Examples

StudentID	Name	Courses
101	Raj	DBMS, OS
102	Priya	DBMS
103	Amit	Thermo, DBMS

- Attribute Courses contains **multiple values**.
- This violates the atomicity rule of 1NF.

- To bring table into 1NF:
 - 1 Break down multi-valued attributes into **separate rows**.
 - 2 Ensure each column contains atomic values only.

Examples

StudentID	Name	Course
101	Raj	DBMS
101	Raj	OS
102	Priya	DBMS
103	Amit	Thermo
103	Amit	DBMS

- Removes **repeating groups** from relation.
- Ensures each field is **atomic**.
- Makes it easier to query and apply relational algebra.
- First step towards reducing anomalies.

Remark

Although 1NF improves structure, it does not remove redundancy completely.

- Still contains **data redundancy**.
- Example:
 - Student 101 enrolled in two courses $\Rightarrow$ student info repeated.
 - If student name changes, it must be updated in multiple rows.
- **Insertion, Updation, Deletion Anomalies** may still exist.

Next Step

To resolve these issues, we move to **Second Normal Form (2NF)**.

- 1NF $\Rightarrow$ Eliminate repeating groups.
- Rule: Each column must hold **single atomic value**.
- Transformation: Split multi-valued attributes into separate rows.
- 1NF ensures:
 - Atomicity of values
 - Clear structure
- But: Redundancy and anomalies still remain.

- 1 Introduction to Normalisation
- 2 First Normal Form (1NF)
- 3 Functional Dependencies**
- 4 Second Normal Form (2NF)
- 5 Third Normal Form (3NF)

- A **Functional Dependency (FD)** is a constraint between two sets of attributes in a relation.
- Formally, for relation R , an FD $X \rightarrow Y$ means:

Definition

If two tuples of R have the same values for attributes X , they must also have the same values for attributes Y .

- Example: In $\text{STUDENT}(\text{StudentID}, \text{Name}, \text{Dept})$,

$$\text{StudentID} \rightarrow \text{Name}, \text{Dept}$$

- $X \rightarrow Y$ X = **determinant**, Y = **dependent**.
- **Left-hand side (LHS)**: attribute(s) that determine others.
- **Right-hand side (RHS)**: attribute(s) that are determined.

Examples

- $\text{StudentID} \rightarrow \text{Name}$ (StudentID uniquely determines Name)
- $\text{CourseID} \rightarrow \text{CourseName}$ (CourseID uniquely determines CourseName)

Examples

R.No	Name	Marks	Dept	Course
1	a	78	CS	C1
2	b	60	EE	C1
3	a	78	CS	C2
4	b	60	EE	C3
5	c	80	IT	C3
6	d	80	EC	C2

- **Valid FDs:**
 - $R.No \rightarrow Name, Marks$
 - $(Name, Marks) \rightarrow Dept$
- **Not Valid:**
 - $Name \rightarrow R.No$
 - $Dept \rightarrow Course$

Definition Reminder

$X \rightarrow Y$: If two tuples agree on X , they must also agree on Y .

Given relation $R(A, B, C, D, E)$:

Examples

A	B	C	D	E
a	2	3	4	5
2	a	3	4	5
a	2	3	6	5
a	2	3	6	6

Check which of the following FDs are valid:

- 1 $A \rightarrow BC$
- 2 $DE \rightarrow C$
- 3 $C \rightarrow DE$
- 4 $BC \rightarrow A$

Given relation $R(A, B, C, D, E)$:

Examples

A	B	C	D	E
a	2	3	4	5
2	a	3	4	5
a	2	3	6	5
a	2	3	6	6

Check which of the following FDs are valid:

- ① $A \rightarrow BC$
- ② $DE \rightarrow C$
- ③ $C \rightarrow DE$
- ④ $BC \rightarrow A$

Answer:

- $A \rightarrow BC$ **Valid**, because same A value gives same B, C .
- $DE \rightarrow C$ **Valid**, same D, E pair gives same C .
- $C \rightarrow DE$ **Not Valid**, same $C = 3$ gives different (D, E) pairs.
- $BC \rightarrow A$ **Valid**, same B, C pair gives same A .

Given relation $R(X, Y, Z)$:

Examples

X	Y	Z
1	4	2
1	5	3
1	6	3
3	2	2

Check which of the following FDs are valid:

- 1 $XY \rightarrow Z \ \&\& \ Z \rightarrow Y$
- 2 $YZ \rightarrow X \ \&\& \ Y \rightarrow Z$
- 3 $YZ \rightarrow X \ \&\& \ X \rightarrow Z$
- 4 $XZ \rightarrow Y \ \&\& \ Y \rightarrow Z$

Given relation $R(A, B, C)$:

Examples

A	B	C
1	2	4
3	5	4
3	7	2
1	4	2

Check which of the following FDs are valid:

- ① $A \rightarrow B \ \&\& \ BC \rightarrow A$
- ② $C \rightarrow B \ \&\& \ CA \rightarrow B$
- ③ $B \rightarrow C \ \&\& \ AB \rightarrow C$
- ④ $A \rightarrow C \ \&\& \ BC \rightarrow A$

- An FD $X \rightarrow Y$ is:
 - ① **Trivial**, if $Y \subseteq X$ Example: $\{\text{StudentID}, \text{Name}\} \rightarrow \text{Name}$
 - ② **Non-Trivial**, if $Y \not\subseteq X$ Example: $\text{StudentID} \rightarrow \text{Dept}$
- Non-trivial FDs are more important for normalisation.

Remark

Trivial FDs always hold, but they do not help in decomposition.

Examples

StudentID	Name	Dept
101	Raj	CSE
102	Priya	CSE
103	Amit	ME

- From table: StudentID $\rightarrow$ Name, Dept
- Dept $\twoheadrightarrow$ StudentID (because many students can be in same Dept)

Examples

Sid	Sname
1	Ranjit
2	Varun

Valid: $Sid \rightarrow Sname$

Examples

Sid	Sname
1	Ranjit
1	Ranjit

Valid: $Sid \rightarrow Sname$

Invalid Cases

Sid	Sname
1	Ranjit
1	Varun

$\Rightarrow$ Invalid, because one Sid determines two different names.

- Axioms define the **rules of inference** for functional dependencies.
- They allow us to derive all possible FDs from a given set.

Core Axioms

- 1 **Reflexivity:** If $Y \subseteq X$, then $X \rightarrow Y$
- 2 **Augmentation:** If $X \rightarrow Y$, then $XZ \rightarrow YZ$
- 3 **Transitivity:** If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

Additional Rules

- ① **Union:** If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$
- ② **Decomposition:** If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$
- ③ **Pseudotransitivity:** If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$
- ④ **Composition:** If $X \rightarrow Y$ and $Z \rightarrow W$, then $XZ \rightarrow YW$

Important

Using these axioms, we can compute **attribute closure** and find **keys**.

- If $X \rightarrow$ all attributes of relation R , then X is a **Candidate Key**.
- Example: In STUDENT(StudentID, Name, Dept)
 - StudentID $\rightarrow$ Name, Dept
 - So **StudentID** is a candidate key.

Important

Functional dependencies help identify **keys**, which are the foundation of higher normal forms.

- FDs describe the **relationship** among attributes.
- They help to:
 - Detect redundancy
 - Identify anomalies
 - Guide decomposition of relations
- Normal forms beyond 1NF (2NF, 3NF, BCNF) are defined using FDs.

- The **closure** of a set of attributes X , written as X^+ , is the set of attributes that can be functionally determined from X using the given FDs.
- Helps us to:
 - Test if X is a **superkey**.
 - Find all candidate keys.
 - Verify decomposition and normalisation.

Notation

If $X \rightarrow Y$, then $Y \in X^+$

- 1 Start with $X^+ = X$.
- 2 For each FD $A \rightarrow B$, if $A \subseteq X^+$, then add B to X^+ .
- 3 Repeat until no more attributes can be added.

Remark

Closure is the main tool to check if X determines all attributes of relation R .

Examples

Relation: $R(A, B, C, D)$ Functional Dependencies:

- $A \rightarrow B$
- $B \rightarrow C$
- $CD \rightarrow A$

Find A^+ :

- 1 Start: $A^+ = \{A\}$
- 2 $A \rightarrow B \Rightarrow A^+ = \{A, B\}$
- 3 $B \rightarrow C \Rightarrow A^+ = \{A, B, C\}$
- 4 $CD \rightarrow A$ (needs D, not satisfied yet)

$A^+ = \{A, B, C\}$

Q1: $R(A, B, C, D, E, F, G)$ FDs:

- $A \rightarrow B$
- $BC \rightarrow DE$
- $AEF \rightarrow G$

Find: $(AC)^+$ **Q2:** $R(A, B, C, D, E)$ FDs:

- $A \rightarrow BC$
- $CD \rightarrow E$
- $B \rightarrow D$
- $E \rightarrow A$

Find: B^+ **Q3:** $R(A, B, C, D, E, F)$ FDs:

- $AB \rightarrow C$
- $BC \rightarrow AD$
- $D \rightarrow E$
- $CF \rightarrow B$

Find: $(AB)^+$ **Q4:** $R(A, B, C, D, E, F, G, H)$ FDs:

- $A \rightarrow BC$
- $CD \rightarrow E$
- $E \rightarrow C$
- $D \rightarrow AEH$
- $ABH \rightarrow BD$
- $DH \rightarrow BC$

Check: $BCD \rightarrow H$?

- To test if X is a **superkey**: Check if X^+ contains all attributes of R .
- To find **candidate keys**: Compute closure of attribute sets and choose minimal keys.
- To validate decompositions in **normalisation**.

Important

Closure connects Functional Dependencies with Keys and Normal Forms.

- **Superkey:** A set of attributes that can **uniquely identify** each tuple in a relation.
- **Candidate Key:** A **minimal superkey**, i.e., no attribute can be removed without losing uniqueness.
- **Primary Key:** One candidate key chosen as the main identifier.

Important

All candidate keys are superkeys, but not all superkeys are candidate keys.

Examples

Relation: STUDENT(RollNo, Name, Dept, Hostel)

FDs:

- RollNo $\rightarrow$ Name, Dept, Hostel
- Dept $\rightarrow$ Hostel

- Superkeys: {RollNo}, {RollNo, Name}, {RollNo, Dept}, {RollNo, Hostel}, ...
- Candidate Key: {RollNo} (minimal)
- If Dept uniquely identified students, then Dept would also be a candidate key.

- Step 1: List all attributes of relation R .
- Step 2: Compute closures using given FDs.
- Step 3: Identify which sets of attributes determine all of $R \Rightarrow$ Superkeys.
- Step 4: Select minimal ones $\Rightarrow$ Candidate Keys.

Examples

Relation: $R(A, B, C)$ FDs: $A \rightarrow B, B \rightarrow C$

- $A^+ = \{A, B, C\} \Rightarrow A$ is a candidate key.
- $B^+ = \{B, C\}$ (not all attributes)
- $C^+ = \{C\}$ (not all attributes)

Examples

Relation: $R(A, B, C)$ FDs:

- $A \rightarrow B$
- $B \rightarrow C$

- $A^+ = \{A, B, C\} \Rightarrow A$ is a superkey.
- No subset of A works $\Rightarrow A$ is a candidate key.
- $B^+ = \{B, C\}$ (not all attributes).
- $C^+ = \{C\}$ (not all attributes).

Candidate Key = $\{A\}$

Examples

Relation: $R(A, B, C, D)$ FDs:

- $AB \rightarrow C$
- $C \rightarrow D$

- $(AB)^+ = \{A, B, C, D\} \Rightarrow AB$ is a superkey.
- Check subsets:
 - $A^+ = \{A\}, B^+ = \{B\}$
 - $C^+ = \{C, D\}, D^+ = \{D\}$
- No single attribute works.

Candidate Key = $\{AB\}$

Examples

Relation: $R(A, B, C, D, E)$ FDs:

- $A \rightarrow BC$
- $CD \rightarrow E$
- $E \rightarrow A$

- $A^+ = \{A, B, C\}$ (not all).
- $(A, D)^+ = \{A, B, C, D, E\} \Rightarrow$ covers all attributes.
- So AD is a superkey.
- Minimality: neither A nor D alone is superkey.

Candidate Key = $\{AD\}$

- 1 Introduction to Normalisation
- 2 First Normal Form (1NF)
- 3 Functional Dependencies
- 4 Second Normal Form (2NF)**
- 5 Third Normal Form (3NF)

- A relation in **1NF** ensures that all attributes contain atomic values.
- But: **Redundancy and anomalies** still exist if some attributes depend only on a part of the primary key.
- To fix this, we move to **Second Normal Form (2NF)**.

Reminder: Candidate Keys

A **candidate key** is the minimal set of attributes that functionally determine all other attributes of the relation.

- A relation is in **Second Normal Form (2NF)** if:
 - ① It is already in **First Normal Form (1NF)**.
 - ② Every non-prime attribute is **fully functionally dependent** on the whole candidate key.

Key Concepts

- **Prime Attribute:** An attribute that is part of some candidate key.
- **Non-Prime Attribute:** An attribute not in any candidate key.
- **Partial Dependency:** When a non-prime attribute depends on **part of a composite key**.
- 2NF eliminates all **partial dependencies**.

Why 2NF?

- Removes redundancy caused by partial dependencies.
- Reduces insertion, updation, and deletion anomalies.

- Let $R(X, Y, Z)$ with candidate key (X, Y) .
- If $X \rightarrow Z$, then Z is dependent on part of the key only.
- This is called a **Partial Dependency**.

Definition

A relation is in **2NF** if:

- 1 It is already in 1NF.
- 2 It has **no partial dependencies** of non-prime attributes on candidate keys.

Examples

Not in 2NF (Partial Dependencies)

Relation: ENROLL(StudentID, CourseID, StudentName, CourseName)

StudentID	CourseID	StudentName	CourseName
101	DB01	Raj	DBMS
101	OS02	Raj	OS
102	DB01	Priya	DBMS

FDs:

- StudentID → StudentName
- CourseID → CourseName

Examples

In 2NF (Decomposed)

STUDENT(StudentID, StudentName)

StudentID	StudentName
101	Raj
102	Priya

COURSE(CourseID, CourseName)

CourseID	CourseName
DB01	DBMS
OS02	OS

ENROLL(StudentID, CourseID)

Examples

Relation: ENROLL(StudentID, CourseID, StudentName, CourseName)

FDs:

- $(\text{StudentID}, \text{CourseID}) \rightarrow \text{StudentName}, \text{CourseName}$
- $\text{StudentID} \rightarrow \text{StudentName}$
- $\text{CourseID} \rightarrow \text{CourseName}$

Why not in 2NF?

- Candidate Key = (StudentID, CourseID)
- Partial Dependencies:
 - $\text{StudentID} \rightarrow \text{StudentName}$
 - $\text{CourseID} \rightarrow \text{CourseName}$
- Non-prime attributes depend on part of key $\rightarrow$ violation of 2NF.

Examples

Relation: ENROLL(StudentID, CourseID, StudentName, CourseName)

FDs:

- $(\text{StudentID}, \text{CourseID}) \rightarrow \text{StudentName}, \text{CourseName}$
- $\text{StudentID} \rightarrow \text{StudentName}$
- $\text{CourseID} \rightarrow \text{CourseName}$

Why not in 2NF?

- Candidate Key = (StudentID, CourseID)
- Partial Dependencies:
 - $\text{StudentID} \rightarrow \text{StudentName}$
 - $\text{CourseID} \rightarrow \text{CourseName}$
- Non-prime attributes depend on part of key $\rightarrow$ violation of 2NF.

Decomposition into 2NF:

- STUDENT(StudentID, StudentName)
- COURSE(CourseID, CourseName)
- ENROLL(StudentID, CourseID)

Examples

Relation: ORDER(OrderID, ProductID, ProductName, CustomerID, CustomerName)

FDs:

- $(\text{OrderID}, \text{ProductID}) \rightarrow \text{ProductName}, \text{CustomerID}, \text{CustomerName}$
- $\text{ProductID} \rightarrow \text{ProductName}$
- $\text{CustomerID} \rightarrow \text{CustomerName}$

Why not in 2NF?

- Candidate Key = $(\text{OrderID}, \text{ProductID})$
- Partial Dependencies:
 - $\text{ProductID} \rightarrow \text{ProductName}$
 - $\text{CustomerID} \rightarrow \text{CustomerName}$

Examples

Relation: ORDER(OrderID, ProductID, ProductName, CustomerID, CustomerName)

FDs:

- (OrderID, ProductID) $\rightarrow$ ProductName, CustomerID, CustomerName
- ProductID $\rightarrow$ ProductName
- CustomerID $\rightarrow$ CustomerName

Why not in 2NF?

- Candidate Key = (OrderID, ProductID)
- Partial Dependencies:
 - ProductID $\rightarrow$ ProductName
 - CustomerID $\rightarrow$ CustomerName

Decomposition into 2NF:

- CUSTOMER(CustomerID, CustomerName)
- PRODUCT(ProductID, ProductName)
- ORDER(OrderID, CustomerID)
- ORDER_DETAIL(OrderID, ProductID)

Examples

Relation: ASSIGN(EmpID, ProjID, EmpName, ProjName, Hours)

FDs:

- $(\text{EmpID}, \text{ProjID}) \rightarrow \text{EmpName}, \text{ProjName}, \text{Hours}$
- $\text{EmpID} \rightarrow \text{EmpName}$
- $\text{ProjID} \rightarrow \text{ProjName}$

Why not in 2NF?

- Candidate Key = $(\text{EmpID}, \text{ProjID})$
- Partial Dependencies:
 - $\text{EmpID} \rightarrow \text{EmpName}$
 - $\text{ProjID} \rightarrow \text{ProjName}$

Examples

Relation: ASSIGN(EmpID, ProjID, EmpName, ProjName, Hours)

FDs:

- $(\text{EmpID}, \text{ProjID}) \rightarrow \text{EmpName}, \text{ProjName}, \text{Hours}$
- $\text{EmpID} \rightarrow \text{EmpName}$
- $\text{ProjID} \rightarrow \text{ProjName}$

Why not in 2NF?

- Candidate Key = (EmpID, ProjID)
- Partial Dependencies:
 - $\text{EmpID} \rightarrow \text{EmpName}$
 - $\text{ProjID} \rightarrow \text{ProjName}$

Decomposition into 2NF:

- EMPLOYEE(EmpID, EmpName)
- PROJECT(ProjID, ProjName)
- ASSIGN(EmpID, ProjID, Hours)

- 2NF removes **partial dependencies**.
- Ensures non-prime attributes depend on the **whole candidate key**.
- Decomposition avoids redundancy, insertion, and deletion anomalies.
- Still: Transitive dependencies may exist $\Rightarrow$ fixed in 3NF.

- ① Relation: RESULT(RollNo, Subject, Name, Marks, SubjectName)
 - Candidate Key = (RollNo, Subject)
 - Identify partial dependencies and decompose to 2NF.
- ② Relation: BOOK_ISSUE(BookID, MemberID, BookTitle, MemberName, IssueDate)
 - Candidate Key = (BookID, MemberID)
 - Find dependencies, then reduce to 2NF.
- ③ Relation: FLIGHT(PilotID, FlightNo, PilotName, FlightName, Hours)
 - Candidate Key = (PilotID, FlightNo)
 - Decompose into 2NF tables.

- 1 Introduction to Normalisation
- 2 First Normal Form (1NF)
- 3 Functional Dependencies
- 4 Second Normal Form (2NF)
- 5 Third Normal Form (3NF)

- A relation is in **Third Normal Form (3NF)** if:
 - ① It is already in **2NF**.
 - ② No **transitive dependency** exists for non-prime attributes.
- **Definition:** For every FD $X \rightarrow Y$ in relation R :
 - Either X is a superkey, OR
 - Y is a prime attribute (part of some candidate key).

Key Idea

3NF ensures that non-prime attributes are **directly dependent** on candidate keys, not indirectly through other attributes.

Examples

FDs:

- $RollNo \rightarrow Dept$
- $Dept \rightarrow HOD$

Then: $RollNo \rightarrow Dept \rightarrow HOD$ $RollNo \rightarrow HOD$ (Transitive)

Visual Representation

$$RollNo \longrightarrow Dept \longrightarrow HOD$$

Non-prime attribute HOD depends on RollNo via Dept.

Examples

Relation: STUDENT(RollNo, Name, Dept, HOD)

FDs:

- RollNo $\rightarrow$ Name, Dept
- Dept $\rightarrow$ HOD

- Candidate Key = RollNo
- 2NF satisfied: No partial dependency (single key).
- But: HOD depends on Dept, which depends on RollNo.
- $\Rightarrow$ **Transitive Dependency present** $\rightarrow$ Not in 3NF.

Examples

Before: STUDENT(RollNo, Name, Dept, HOD)

Problems:

- HOD repeats for every student in the same Dept.
- Updating HOD requires multiple changes.
- Deleting all students of Dept removes HOD info.

Examples

After (3NF):

- STUDENT(RollNo, Name, Dept)
- DEPARTMENT(Dept, HOD)

Examples

Relation: ISSUE(BookID, Title, Author, MemberID, MemberName)

FDs:

- BookID $\rightarrow$ Title, Author
- MemberID $\rightarrow$ MemberName

Examples

Relation: ISSUE(BookID, Title, Author, MemberID, MemberName)

FDs:

- BookID $\rightarrow$ Title, Author
- MemberID $\rightarrow$ MemberName

Why not in 3NF?

- Candidate Key = (BookID, MemberID)
- MemberName depends on MemberID (non-prime) $\rightarrow$ transitive wrt key.

Examples

Relation: ISSUE(BookID, Title, Author, MemberID, MemberName)

FDs:

- BookID $\rightarrow$ Title, Author
- MemberID $\rightarrow$ MemberName

Why not in 3NF?

- Candidate Key = (BookID, MemberID)
- MemberName depends on MemberID (non-prime) $\rightarrow$ transitive wrt key.

Decomposition:

- BOOK(BookID, Title, Author)
- MEMBER(MemberID, MemberName)
- ISSUE(BookID, MemberID)

2NF

- Removes partial dependencies.
- Non-prime attributes depend on the **whole key**.
- Still may have transitive dependencies.

3NF

- Already in 2NF.
- Removes transitive dependencies.
- Non-prime attributes must depend **directly** on candidate key.

Summary

2NF → No partial dependencies 3NF → No partial + no transitive dependencies

- ① Relation: EMP(EmpID, EmpName, DeptID, DeptName, Location)
- FDs: EmpID $\rightarrow$ EmpName, DeptID
 - DeptID $\rightarrow$ DeptName, Location
 - Decompose into 3NF.

- ➊ Relation: EMP(EmpID, EmpName, DeptID, DeptName, Location)
 - FDs: EmpID $\rightarrow$ EmpName, DeptID
 - DeptID $\rightarrow$ DeptName, Location
 - Decompose into 3NF.
- ➋ Relation: MOVIE(MovieID, Title, Director, DirPhone)
 - FDs: MovieID $\rightarrow$ Title, Director
 - Director $\rightarrow$ DirPhone
 - Decompose into 3NF.

Thank You!



Normalisation - 2

DBMS Lecture Series

Unit-3

Pavinder Yadav

Assistant Professor
School of Computer Science
University of Petroleum and Energy Studies
Dehradun



- 1 Boyce–Codd Normal Form (BCNF)
- 2 Multivalued Dependencies (MVD)
- 3 Fourth Normal Form (4NF)
- 4 Fifth Normal Form (5NF)

- 1 Boyce–Codd Normal Form (BCNF)
- 2 Multivalued Dependencies (MVD)
- 3 Fourth Normal Form (4NF)
- 4 Fifth Normal Form (5NF)

- A relation is in **BCNF** if:
 - 1 It is already in 3NF.
 - 2 For every non-trivial FD $X \rightarrow Y$ in R , X must be a superkey.

Key Difference from 3NF

3NF allows FDs where Y is a prime attribute, but BCNF requires the LHS (X) of every FD to be a superkey.

Important

BCNF is stricter than 3NF and removes certain anomalies still possible in 3NF.

Examples

Relation: TEACHER(TeacherID, Course, Textbook)

FDs:

- $(\text{TeacherID}, \text{Course}) \rightarrow \text{Textbook}$
- $\text{Textbook} \rightarrow \text{Course}$

Candidate Key = (TeacherID, Course)

- This relation is in 3NF:
 - All non-prime attributes depend on candidate key.
 - $\text{Textbook} \rightarrow \text{Course}$ is allowed in 3NF since Course is prime.
- But: *Textbook* is not a superkey $\rightarrow$ violates BCNF.
- $\Rightarrow$ Some redundancy remains.

- ① Ensure relation is already in 3NF.
- ② Find candidate keys (using closures).
- ③ For each FD $X \rightarrow Y$:
 - Check if X is a superkey.
 - If not $\rightarrow$ violation of BCNF.
- ④ Decompose relation into two smaller relations:
 - One containing $X \cup Y$.
 - Another containing attributes of $R - (Y - X)$.
- ⑤ Repeat until all relations satisfy BCNF.

Result

Final decomposition is always in BCNF and free of redundancy.

Relation: ENROLL(StudentID, Course, Instructor)

FDs:

- StudentID, Course $\rightarrow$ Instructor
- Instructor $\rightarrow$ Course

Candidate Key = (StudentID, Course)

Step 1: Already in 3NF.

Step 2: Check FD *Instructor* $\rightarrow$ *Course*

- LHS = Instructor is not a superkey.
- $\Rightarrow$ Violation of BCNF.

Decomposition:

- R1(Instructor, Course)
- R2(StudentID, Instructor)

Relation: SUPPLY(Supplier, Part, City)

FDs:

- Supplier, Part $\rightarrow$ City
- City $\rightarrow$ Supplier

Candidate Key = (Supplier, Part)

Step 1: In 3NF.

Step 2: Check FD *City $\rightarrow$ Supplier*

- LHS = City is not a superkey.
- $\Rightarrow$ Not in BCNF.

Decomposition:

- R1(City, Supplier)
- R2(City, Part)

3NF

- Already in 2NF.
- Allows FDs where LHS is not superkey if RHS is prime.
- Removes partial + transitive dependencies.
- Still may have redundancy in some cases.

BCNF

- Already in 3NF.
- Requires LHS of every FD to be a **superkey**.
- Stricter than 3NF.
- Eliminates almost all redundancy and anomalies.

Summary

3NF Practical use BCNF = Stronger theoretical guarantee

- 1 Relation: CLASS(Student, Course, Teacher)
 - FDs: (Student, Course) $\rightarrow$ Teacher
 - Teacher $\rightarrow$ Course
- 2 Relation: GAME(Player, Team, Coach)
 - FDs: Player $\rightarrow$ Team
 - Team $\rightarrow$ Coach
- 3 Relation: BOOK(BookID, Author, AuthorCountry)
 - FDs: BookID $\rightarrow$ Author
 - Author $\rightarrow$ AuthorCountry

Task

For each relation:

- Find candidate keys.
- Check BCNF condition.
- If violated, decompose into BCNF relations.

- 1 Boyce–Codd Normal Form (BCNF)
- 2 Multivalued Dependencies (MVD)
- 3 Fourth Normal Form (4NF)
- 4 Fifth Normal Form (5NF)

Definition

In a relation R , a multivalued dependency $X \twoheadrightarrow Y$ holds if:

For each value of X , the set of values of Y is independent of the rest of the attributes.

- Unlike FDs, MVDs describe situations where attributes are **independent but repeat together**.
- Denoted by $X \twoheadrightarrow Y$.
- Every FD $X \rightarrow Y$ is also an MVD, but not vice versa.

Intuition

If $X \twoheadrightarrow Y$, then knowing X gives a set of possible Y values, regardless of other attributes.

Relation: STUDENT(StudentID, Hobby, Language)

- A student can have multiple hobbies.
- A student can also know multiple languages.
- Hobbies and Languages are **independent of each other**.

$StudentID \twoheadrightarrow Hobby$

$StudentID \twoheadrightarrow Language$

Problem

If stored in one table, hobbies $\times$ languages combinations appear (unnecessary repetition).

Examples

Not in 4NF (Redundancy)

STUDENT(StudentID, Hobby, Language)

StudentID	Hobby	Language
101	Cricket	English
101	Cricket	Hindi
101	Chess	English
101	Chess	Hindi

MVDs:

- StudentID $\twoheadrightarrow$ Hobby
- StudentID $\twoheadrightarrow$ Language

Examples

After 4NF Decomposition

STUDENT_HOBBY(StudentID, Hobby)

StudentID	Hobby
101	Cricket
101	Chess

STUDENT_LANG(StudentID, Language)

StudentID	Language
101	English
101	Hindi

Examples

Not in 4NF (Redundancy)

BOOK(BookID, Author, Genre)

BookID	Author	Genre
B1	A. Sharma	Fiction
B1	A. Sharma	Thriller
B1	R. Gupta	Fiction
B1	R. Gupta	Thriller

MVDs:

- BookID $\twoheadrightarrow$ Author
- BookID $\twoheadrightarrow$ Genre

Examples

After 4NF Decomposition

BOOK_AUTHOR(BookID, Author)

BookID	Author
B1	A. Sharma
B1	R. Gupta

BOOK_GENRE(BookID, Genre)

BookID	Genre
B1	Fiction
B1	Thriller

- Data redundancy (Cartesian product of independent sets).
- Insertion anomaly: To add a new hobby, we must also pair it with all known languages.
- Deletion anomaly: Deleting one hobby-language pair may remove info about a language.
- Update anomaly: Changing a hobby requires updating multiple tuples.

Examples

STUDENT(StudentID=101):

- Hobbies = Cricket, Chess
- Languages = English, Hindi

Stored together → 4 tuples (Cricket-English, Cricket-Hindi, Chess-English, Chess-Hindi).

- 1 Boyce–Codd Normal Form (BCNF)
- 2 Multivalued Dependencies (MVD)
- 3 Fourth Normal Form (4NF)**
- 4 Fifth Normal Form (5NF)

- A relation is in **4NF** if:
 - 1 It is already in **BCNF**.
 - 2 For every non-trivial MVD $X \twoheadrightarrow Y$, X must be a superkey.

Non-trivial MVD

$X \twoheadrightarrow Y$ is non-trivial if:

- $Y \not\subseteq X$, and
- $X \cup Y \neq R$

Summary

4NF eliminates redundancy caused by **multivalued dependencies**.

Relation: STUDENT(StudentID, Hobby, Language)

MVDs:

- StudentID $\twoheadrightarrow$ Hobby
- StudentID $\twoheadrightarrow$ Language

Step 1: Candidate Key = (StudentID, Hobby, Language)

Step 2: MVD Check

- StudentID is not a superkey.
- $\Rightarrow$ Not in 4NF.

Step 3: Decomposition

- STUDENT_HOBBY(StudentID, Hobby)
- STUDENT_LANG(StudentID, Language)

Now each relation is in 4NF.

Relation: BOOK(BookID, Author, Genre)

MVDs:

- BookID $\twoheadrightarrow$ Author
- BookID $\twoheadrightarrow$ Genre

Why Not in 4NF?

- Candidate Key = (BookID, Author, Genre)
- BookID is not a superkey but determines independent sets.

Decomposition:

- BOOK_AUTHOR(BookID, Author)
- BOOK_GENRE(BookID, Genre)

Now relation is in 4NF.

BCNF

- Deals with **Functional Dependencies (FDs)**.
- Requires every FD $X \rightarrow Y$ to have X as superkey.
- Removes redundancy from transitive dependencies.

4NF

- Deals with **Multivalued Dependencies (MVDs)**.
- Requires every MVD $X \twoheadrightarrow Y$ to have X as superkey.
- Removes redundancy from independent multivalued facts.

Summary

BCNF $\rightarrow$ FDs

4NF $\rightarrow$ FDs + MVDs

- 1 Relation: PLAYER(PlayerID, Sport, Country)
 - PlayerID $\rightarrow$ Sport
 - PlayerID $\rightarrow$ Country
- 2 Relation: MOVIE(MovieID, Actor, Language)
 - MovieID $\rightarrow$ Actor
 - MovieID $\rightarrow$ Language
- 3 Relation: CAR(CarModel, Feature, Dealer)
 - CarModel $\rightarrow$ Feature
 - CarModel $\rightarrow$ Dealer

Task

For each relation:

- Find candidate keys.
- Identify MVDs.
- Check if in 4NF.
- If not, decompose into 4NF relations.

- 1 Boyce–Codd Normal Form (BCNF)
- 2 Multivalued Dependencies (MVD)
- 3 Fourth Normal Form (4NF)
- 4 Fifth Normal Form (5NF)**

Definition

A Join Dependency (JD) $JD(R_1, R_2, \dots, R_n)$ on a relation R means:

$$R = \pi_{R_1}(R) \bowtie \pi_{R_2}(R) \bowtie \dots \bowtie \pi_{R_n}(R)$$

That is, R can be reconstructed by joining its projections on subsets of attributes.

- FDs and MVDs are special cases of JDs.
- JDs describe situations where a relation can be split into 3 or more tables and then losslessly joined back.

Intuition

A JD exists when a table contains data that can be reconstructed perfectly by joining smaller projections.

- A relation is in **5NF** (also called PJNF) if:
 - 1 It is already in 4NF.
 - 2 Every non-trivial Join Dependency (JD) in R is implied by the candidate keys.

Non-trivial JD

JD is non-trivial if it requires splitting R into 3 or more relations that are not implied by FDs or MVDs.

Key Idea

5NF eliminates redundancy that arises from **join dependencies**.

Examples

Relation: SUPPLY(Supplier, Part, Project)

Facts:

- Supplier S1 supplies Part P1 for Project J1.
- Supplier S1 supplies Part P1 for Project J2.
- Supplier S1 supplies Part P2 for Project J1.
- Supplier S1 supplies Part P2 for Project J2.

- No FD or MVD exists among attributes.
- But redundancy exists:
 - Knowing (Supplier, Part) and (Supplier, Project) and (Part, Project) allows reconstruction.
- This is a case of a **Join Dependency**.

Relation: SUPPLY(Supplier, Part, Project)

JD: SUPPLY = $\pi_{Supplier, Part}(SUPPLY) \bowtie \pi_{Supplier, Project}(SUPPLY) \bowtie \pi_{Part, Project}(SUPPLY)$

Step 1: Check 4NF

- No FDs or MVDs $\rightarrow$ relation is already in 4NF.

Step 2: Check JD

- A non-trivial JD exists $\rightarrow$ not in 5NF.

Step 3: Decomposition

- SUP1(Supplier, Part)
- SUP2(Supplier, Project)
- SUP3(Part, Project)

Now relation is in 5NF.

4NF

- Removes redundancy due to **MVDs**.
- Condition: For every non-trivial MVD $X \twoheadrightarrow Y$, X must be a superkey.
- Does not handle redundancy due to JDs.

5NF

- Removes redundancy due to **Join Dependencies**.
- Condition: For every JD, each determinant must be a superkey.
- Ensures complete elimination of all redundancy.

Summary

4NF $\rightarrow$ FDs + MVDs

5NF $\rightarrow$ FDs + MVDs + JDs

Examples

Not in 5NF (Redundant)

SUPPLY(Supplier, Part, Project)

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S1	P2	J2

No FD or MVD exists, but redundancy remains.

Examples

Decomposition into 5NF

R1(Supplier, Part)

Supplier	Part
S1	P1
S1	P2

R2(Supplier, Project)

Supplier	Project
S1	J1
S1	J2

R3(Part, Project)

Part	Project
P1	J1
P1	J2
P2	J1
P2	J2

Examples

Not in 5NF (Redundant)

COURSE(Course, Teacher, Textbook)

Course	Teacher	Textbook
DBMS	Sharma	Book1
DBMS	Sharma	Book2
DBMS	Verma	Book1
DBMS	Verma	Book2

No FD or MVD exists, but Cartesian product redundancy appears.

Examples

Decomposition into 5NF

R1(Course, Teacher)	Course	Teacher
	DBMS	Sharma
	DBMS	Verma

R2(Course, Textbook)	Course	Textbook
	DBMS	Book1
	DBMS	Book2

Examples

Not in 5NF (Redundant)

ASSIGN(Project, Employee, Skill)

Project	Employee	Skill
P1	E1	Java
P1	E1	SQL
P1	E2	Java
P1	E2	SQL

No FD or MVD, but redundancy from combinations.

Examples

Decomposition into 5NF

R1(Project, Employee)

Project	Employee
P1	E1
P1	E2

R2(Project, Skill)

Project	Skill
P1	Java
P1	SQL

R3(Employee, Skill)

Employee	Skill
E1	Java
E1	SQL
E2	Java
E2	SQL

- ① Relation: PROJECT(ProjectID, Employee, Skill)
 - Facts: Employees can have multiple skills, and projects require multiple employees.
 - Check if decomposition into (ProjectID, Employee), (ProjectID, Skill), (Employee, Skill) is needed.
- ② Relation: COURSE(CourseID, Teacher, Textbook)
 - Facts: Each course can have multiple teachers, multiple textbooks.
 - Check for JD and decompose into 5NF.
- ③ Relation: EVENT(EventID, Guest, Sponsor)
 - Facts: Multiple guests and sponsors are independent.
 - Identify join dependency and perform decomposition.

Multivalued Dependency (MVD)

- An MVD $X \twoheadrightarrow Y$ means that for each value of X , the set of values of Y is independent of the other attributes in the relation.
- It captures situations with **two independent facts** about the same entity.
- Example: A student can have multiple hobbies and multiple languages, both independent of each other.
- Decomposition: Relation is split into two smaller tables.

Join Dependency (JD)

- A JD $JD(R_1, R_2, \dots, R_n)$ means a relation R can be reconstructed exactly by joining $n \geq 3$ projections.
- It captures situations where **three or more facts are independent**.
- Example: Supplier–Part–Project, where each pair gives enough info to reconstruct the whole relation.
- Decomposition: Relation is split into three or more smaller tables.

Aspect	Multivalued Dependency (MVD)	Join Dependency (JD)
Definition	$X \twoheadrightarrow Y$: values of Y are independent of rest given X	$R = \pi_{R_1}(R) \bowtie \pi_{R_2}(R) \dots \pi_{R_n}(R), n \geq 3$
Facts captured	Two independent sets of attributes	Three or more independent sets of attributes
Minimum Projections	Always 2	At least 3
Special Case	Every FD is also an MVD	Every MVD is a special case of JD
Example	STUDENT(StudentID, Hobby, Language)	SUPPLY(Supplier, Part, Project)
Decomposition	Split into 2 relations	Split into 3 or more relations
Normal Form	Eliminated in 4NF	Eliminated in 5NF

Summary of Normal Forms (1NF – 5NF)

1NF

- Eliminate repeating groups.
- Each attribute must have **atomic values**.

2NF

- Already in 1NF.
- Remove **partial dependencies**.
- Every non-prime attribute depends on the **whole key**.

3NF

- Already in 2NF.
- Remove **transitive dependencies**.
- Non-prime attributes must depend directly on candidate keys.

BCNF

- Stronger than 3NF.
- For every FD $X \rightarrow Y$, X must be a **superkey**.

4NF

- Already in BCNF.
- Eliminate redundancy due to **multivalued dependencies (MVDs)**.
- For every $X \twoheadrightarrow Y$, X must be a superkey.

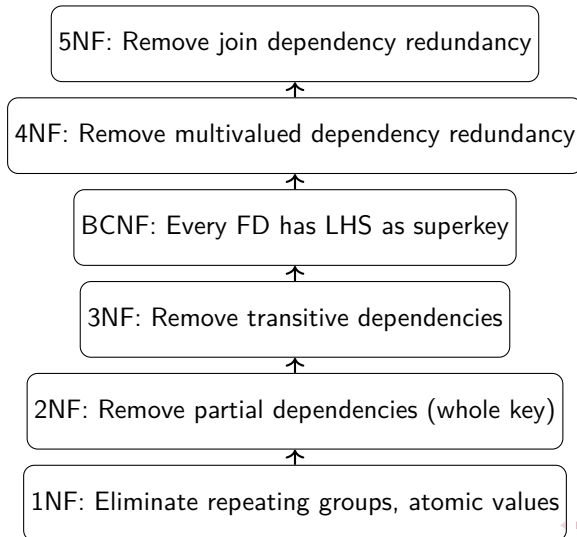
5NF

- Already in 4NF.
- Eliminate redundancy due to **join dependencies (JDs)**.
- Relation should not need decomposition into 3+ tables unless implied by keys.

Quick Recap

- 1NF → Atomic values
- 2NF → No partial dependencies
- 3NF → No transitive dependencies
- BCNF → Strong FD rule (LHS must be superkey)
- 4NF → No MVD redundancy
- 5NF → No JD redundancy

The Normalisation Ladder (1NF – 5NF)



Thank You!

